Дообучение GPT-2-рекомендера с подкреплением по обученной модели награды

Отчёт, версия 2: Yambda-50M и KION, две итерации

Даниил Антонов 11 октября 2026

Рабочий отчёт. Числа получены в пяти завершённых запусках (Таблица 2).

Кратко. Рекомендер на архитектуре GPT-2 дообучался градиентом политики, чтобы повысить долю прослушивания, которую предсказывает второй GPT-2 — модель награды. Конвейер отработал пять раз: на Yambda-50M с REINFORCE (дважды) и с PPO, на KION с PPO (дважды; второй раз — с исследованием наград). Понятия и формулы объяснены в отдельном документе «Теоретический минимум».

1. **Базовые модели в порядке.** На Yambda GPT-рекомендер — лучший из сравниваемых (NDCG@10 0.114 против 0.110 у GRU4Rec, 0.105 у BERT4Rec, 0.077 у SASRec и 0.019 у популярного). На KION он в середине (0.122; BERT4Rec 0.129, популярное 0.090). AUC модели награды — 0.87 на Yambda и 0.75 на KION.
2. **Без якоря политика разрушается всегда:** на обоих наборах и с обоими оптимизаторами все пользователи получают одни и те же объекты.
3. **С якорем результат зависит от набора данных.** На KION при весе KL 0.3 предсказанная доля просмотра растёт на 23%, релевантность на досмотренном сохраняется (+1%), Recall на брошенном падает на 20%; результат воспроизвёлся с заново обученными моделями награды. На Yambda с PPO при том же весе почти ничего не меняется: награда +1%, релевантность +2%.
4. **Порог коллапса задаёт способ оценки преимущества, а не набор данных.** На Yambda при одном и том же весе KL 0.1 REINFORCE теряет 89% NDCG@10, а PPO не теряет ничего. Веса KL между постановками несравнимы.
5. **Награда из лога удерживает релевантность без якоря, но не разнообразие.** Награда статьи [1] — попадание в реальное продолжение — сохраняет NDCG@10 (+0.6%) при нулевом якоре. В сумме с долей просмотра она даёт +39% награды при прежней релевантности на досмотренном, что лучше якоря при той же награде. Но списки стягиваются к популярному: покрытие каталога падает с 0.50 до 0.10.
6. **Голова выбора не сработала.** Модель награды отличает реальный объект от подменённого (AUC 0.70), но среди кандидатов рекомендера знает о релевантности гораздо меньше самого рекомендера (0.56 против 0.70). Политики с нулевой релевантностью получают по ней оценку выше базовой.
7. **Возвращение предсказуемо, но это свойство пользователя, а не объекта.** AUC 0.70–0.72, однако между кандидатами одного пользователя сигнал почти не различается. Награда за него даёт +1.9 пункта вероятности возвращения за 7% NDCG@10. Награда за продолжение сессии вредна: она поощряет тайтлы, которые бросают.
8. **Для умеренного выигрыша RL не нужен.** На KION переранжирование не уступает RL до роста награды примерно на треть; дообучение на понравившемся и REINFORCE вне политики из [2] попадают в точку основного запуска RL; отбор лучшего из нескольких списков слабее всех.

Не сделано: несколько начальных значений; исследование наград на Yambda.

1 Задача и постановка

В задаче три шага: обучить рекомендер на архитектуре GPT-2 предсказывать следующий объект, где токены — идентификаторы объектов; построить модель награды, которая по истории предсказывает реакцию пользователя на рекомендованный объект; дообучить рекомендер с помощью RL в сторону последовательностей с более высокой ожидаемой наградой по оценке этой модели. Работа опирается на две статьи. Петров и Макдональд [1] дообучают GPTRec алгоритмом PPO по награде, в которой NDCG сочетается с разнообразием или штрафом за популярность. Матвеев и соавторы [2] обучают модель отбора кандидатов на Yambda-5B алгоритмом REINFORCE вне политики на залогированных сессиях, а обученную модель обратной связи используют для оценки.

1.1 Данные

Первый набор — Yambda-50M [3], лог прослушиваний Яндекс Музыки: 46.5 млн прослушиваний 10 000 пользователей примерно за 300 дней. У каждого события есть трек, время, прослушанная доля трека, длина трека и признак того, выбрал ли пользователь трек сам или его подал рекомендер. Предобработка делает следующее.

- **Сессии.** Событие «продолжает сессию», если у того же пользователя есть другое событие в пределах 30 минут.
- **Разбиение.** Одна общая ось времени: последние 7 дней — тест, 7 дней перед ними — валидация, остальное — обучение. Всё, что выводится из данных (каталог, популярность, статистика объектов), считается только по обучающему периоду.
- **Каталог.** Самые популярные треки обучающего периода становятся токенами, пронумерованными по популярности; события с другими треками отбрасываются. Пользователю нужно не меньше 50 обучающих событий.

При каталоге в 50 000 треков остаётся 37.2 млн событий (80.1%) 8 711 пользователей: 35.0 млн в обучении, 1.13 млн в валидации и 1.08 млн в тесте. Средняя доля прослушивания — 0.65; 64% событий прослушаны хотя бы наполовину; 95% событий продолжают сессию; 51% событий органические. Распределение доли двугорбое (Рис. 1): трек либо пропускают в начале, либо слушают до конца, поэтому модель награды по сути классифицирует пропуски. Короткие треки дослушивают чуть чаще длинных: 0.70 против 0.60 между крайними децилями длительности; от популярности доля заметно не зависит.

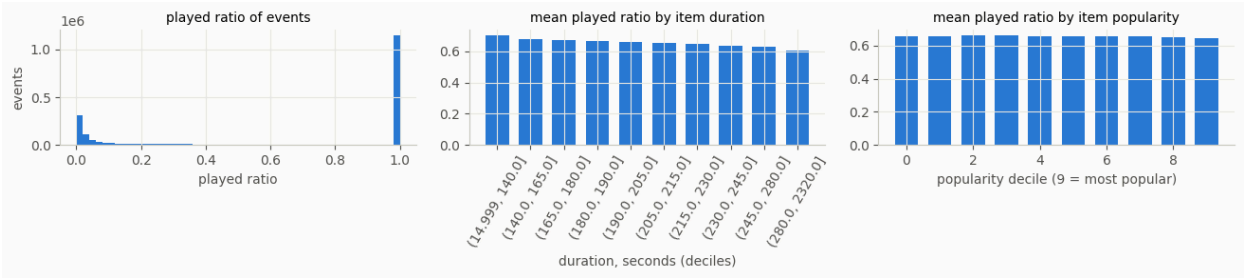


Рис. 1. Сигнал награды в обработанном логе Yambda (каталог 50 000 треков). Слева: распределение доли прослушивания. В центре и справа: средняя доля по децилям длительности и популярности треков.

Второй набор — KION [4], лог онлайн-кинотеатра MTC: 5.5 млн строк 962 тыс. пользователей примерно за пять месяцев. От Yambda он отличается в том, что важно на каждом шаге. На пару «пользователь, тайтл» приходится одна строка: дата последнего просмотра, суммарное время просмотра и просмотренная доля тайтла; эта доля и служит здесь долей потребления. Время задано датами. Нет признака, был ли тайтл рекомендован. Истории короткие. Предобработка та же, с подходящими порогами: тайтлу нужно 10 обучающих событий, пользователю 5, а событие «продолжает сессию», если пользователь посмотрел другой тайтл в течение 7 дней. Таблица 1 сопоставляет два обработанных лога.

Таблица 1. Два обработанных лога.

	Yambda-50M (запуск A)	**KION**
Оставлено событий	37.2 млн (80.1%)	3.88 млн (70.9%)
Пользователей	8 711	256 407
Объектов в каталоге	50 000	7 992
Событий на пользователя, в среднем	4 270	15
События: обучение / валидация / тест	35.0 / 1.13 / 1.08 млн	3.51 / 0.19 / 0.18 млн
Средняя доля потребления	0.65	0.50
Событий с долей не меньше половины	64%	48%
Событий, продолжающих сессию	95% (30 минут)	80% (7 дней)
Тестовых пользователей с запросом	7 593	59 997

Доля просмотра на KION тоже двугорбая (Рис. 2), но зависит от длины тайтла гораздо сильнее, чем доля прослушивания трека. Тайтлы длиной с фильм, примерно от 80 до 140 минут, досматривают в среднем до 0.5. Самую длинную десятую часть каталога — больше 8.5 часов содержимого, то есть сериалы — досматривают до 0.22, потому что доля считается от всего сериала. Награда, построенная на этой доле, предпочитает фильмы ещё до всякой модели.

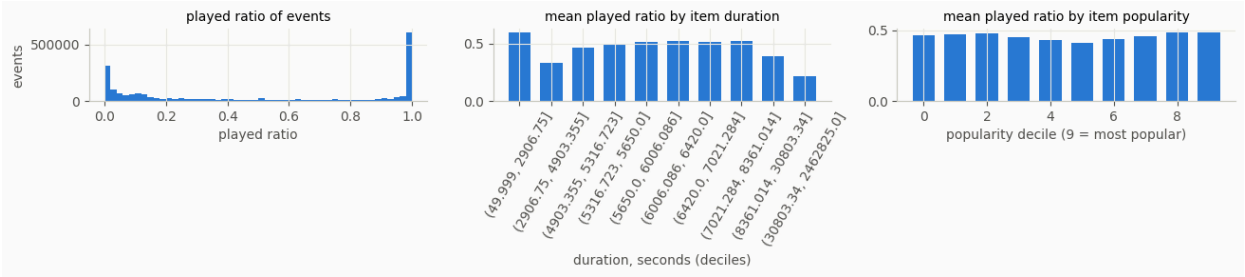


Рис. 2. Сигнал награды в обработанном логе KION. Слева: распределение доли просмотра. В центре и справа: средняя доля по децилям длительности и популярности тайтлов.

1.2 Модели

Базовый рекомендер. Декодер GPT-2 (реализация Hugging Face, 4 слоя, ширина 256, 4 головы внимания, dropout 0.1) над токенами объектов, со связанными входными и выходными эмбеддингами. Он обучается на окнах из 128 подряд идущих событий одного пользователя кросс-энтропией с полным softmax по следующему объекту, оптимизатор AdamW со скоростью обучения 10^{-3} и косинусным затуханием, ранняя остановка по валидационным потерям. Рекомендации он выдаёт двумя способами. *Ранжирование* (top-k) один раз оценивает все объекты и берёт лучшие. *Генерация* (next-k) порождает десять объектов по одному, подавая каждый выбор обратно на вход и не повторяя объект внутри списка. RL меняет второй способ. На KION окна длиной 48 событий, dropout 0.2, а тайтлы из истории пользователя не рекомендуются никогда, потому что каждая пара «пользователь, тайтл» встречается один раз.

Модель награды. Второй декодер GPT-2 того же размера. Токен на позиции t несёт объект t вместе с долей потребления объекта $t-1$ (в десяти корзинах), поэтому выход на позиции t видел объект-кандидат и всю историю, но не реакцию, которую он предсказывает. Две головы предсказывают долю потребления (кросс-энтропия с долей в роли мягкой метки) и продолжение сессии. Обучаются две копии на непересекающихся половинах пользователей. *Обучающая* модель награды выдаёт награды во время RL. *Отложенная* никогда не влияет на политику и только оценивает её списки: выигрыш, который она не подтверждает, — это использование ошибок первой.

RL с REINFORCE (запуски A и B). На каждом шаге берётся батч моментов рекомендации из обучающих историй (пользователь и точка обрезки), и для каждого текущая политика порождает несколько списков по десять треков. Обучающая модель награды предсказывает долю прослушивания каждого трека каждого списка, позиция за позицией; для порождённых треков она подаёт себе на вход собственную ожидаемую долю как их обратную связь. Награда трека

— его предсказанная доля, а возврат — сумма наград от его позиции до конца списка. Базовая линия списка — средний возврат других списков для той же истории, что оставляет градиент несмещённым. Преимущества приводятся к единичной дисперсии. Потери:

$$\mathcal{L}(\theta) = -\mathbb{E}[A \cdot \log \pi_\theta(a \mid s)] + \beta \cdot \mathrm{KL}(\pi_\theta \parallel \pi_0),$$

где π_0 — замороженный базовый рекомендер, а KL считается точно по всему каталогу на каждой позиции. Это REINFORCE с базовой линией и KL-якорем. В награде нет слагаемого релевантности, поэтому вес якоря β — единственное, что требует от политики оставаться релевантной.

RL с PPO (запуск C). Базовую линию заменяет модель ценности: копия трансформера рекомендера со скалярной головой, обученная предсказывать возврат из каждой позиции списка. Преимущества считаются обобщённой оценкой (дисконт 0.99, $\lambda = 0.9$) и приводятся к единичной дисперсии. На каждой итерации порождается по одному списку для 512 историй и делается четыре шага градиента по обрезанной целевой функции PPO (порог 0.2); добавляется тот же KL-якорь. Скорость обучения политики 10^{-5}, модели ценности 10^{-4}; первые 50 итераций обучается только модель ценности. Dropout при дообучении выключен.

Бейзлайны (запуск C). Самые популярные объекты, цепь Маркова первого порядка, GRU4Rec с полным softmax, SASRec с исходной функцией потерь (бинарная кросс-энтропия с одним случайным отрицательным примером на позицию) и BERT4Rec, обученный восстанавливать замаскированные объекты. Нейросетевые модели берут ширину, глубину, dropout и длину окна рекомендера, обучаются на тех же окнах и останавливаются по своим валидационным потерям. Ни одна не настраивалась.

Альтернативы RL (запуск C). Два более дешёвых способа поднять награду. *Дообучение только на понравившемся* продолжает обычное обучение рекомендера одну эпоху, но только к объектам, потреблённым хотя бы наполовину; модель награды не используется. *Переранжирование* берёт 100 лучших кандидатов рекомендера и упорядочивает их по сумме $w \cdot$ (логарифм вероятности по рекомендеру) + (награда по обучающей модели награды) для нескольких весов w; обучения не требуется. Награда каждого кандидата предсказывается как для следующего объекта после реальной истории.

1.3 Оценка

Каждый тестовый пользователь с историей делает один запрос в начале тестовой недели: последние 118 событий (38 на KION) — история, следующие не более чем 20 событий — правда. Таких пользователей 7 593 на Yambda при каталоге в 50 000 треков и 59 997 на KION. Измеряются три группы величин.

- **Релевантность.** Recall, NDCG, MAP и HitRate на 1, 5 и 10 против всех 20 будущих объектов, против потреблённых хотя бы наполовину (*liked*) и против остальных (*skipped*). Этому разбиению не нужна модель награды: политика, которая на самом деле научилась избегать пропусков, должна терять релевантность на *skipped* и сохранять на *liked*.
- **Предсказанная награда.** Обе модели награды оценивают порождённые списки.
- **Коллапс.** Покрытие каталога, доля пользователей, получивших самый частый объект, ожидаемое пересечение списков двух пользователей, доля рекомендаций, уже бывших в истории, популярность и длительность рекомендованных объектов.

В запуске C изменение от базового рекомендера к дообученному приводится с 95%-м интервалом парного бутстрапа по пользователям. Он измеряет шум выборки пользователей, а не шум между запусками обучения.

1.4 Пять запусков

Таблица 2 показывает, чем они различались. Запуск B рассчитан на гораздо более слабую видеокарту; помимо каталога, это запуск A с меньшим объёмом обучения. Запуск C первым использовал PPO, бейзлайны, альтернативы RL и интервалы. Запуск D повторяет его конвейер

на Yambda и тем самым отделяет набор данных от оптимизатора. Запуск E повторяет запуск C с новыми моделями награды и добавляет исследование наград (раздел 5).

Таблица 2. Пять завершённых запусков. Везде горизонт 10 объектов, окна по 128 событий на Yambda и по 48 на KION.

	A	**B**	**C**	**D**	**E**
Набор данных	Yambda-50M	Yambda-50M	KION	Yambda-50M	KION
Видеокарта	RTX PRO 6000	Tesla T4	RTX PRO 6000	RTX PRO 6000	RTX PRO 6000
Каталог	50 000	30 000	7 992	50 000	7 992
Пользователей	8 711	8 669	256 407	8 711	256 407
Эпох рекомендера	15	6	11 (ранняя остановка)	10	11 (ранняя остановка)
Алгоритм RL	REINFORCE	REINFORCE	PPO	PPO	PPO
Основной запуск RL	4 000 × 1024 списка	1 500 × 256	2 000 × 512	2 000 × 512	2 000 × 512
Списков на историю	4	4	1	1	1
Запуски перебора	2 000 шагов	500 шагов	1 000 итераций	1 000 итераций	1 000 итераций
Вес KL, основной	0.1	0.1	0.3	0.3	0.3
Веса KL в переборе	0, 0.01, 0.03, 0.1, 0.3, 1, 3	0, 0.03, 0.3, 3	0, 0.1, 0.2, 0.3, 0.5, 1	0, 0.1, 0.2, 0.3, 0.5, 1	0, 0.1, 0.2, 0.3, 0.5, 1
Скорость обучения RL	$3 \cdot 10^{-5}$	$3 \cdot 10^{-5}$	10^{-5}	10^{-5}	10^{-5}
Бейзлайны, альтернативы, интервалы	нет	нет	да	да	да
Исследование наград	нет	нет	нет	нет	да
Время работы	9.5 ч	5.0 ч	1.3 ч	6.0 ч	3.1 ч

2 Yambda-50M, REINFORCE (запуски A и B)

2.1 Базовый рекомендер

Таблица 3 показывает рекомендер после обычного обучения. Он намного выше популярного по всем мерам. В отличие от результата Петрова и Макдональда [1], генерация списка объект за объектом не вредит: NDCG@10 даже немного выше, чем при ранжировании. У них правда — один следующий объект, и каждая следующая позиция порождённого списка обусловлена неверным началом. Здесь правда — следующие 20 событий, и модель, обученная продолжать последовательности, делает именно то, что от неё требуется. Половина того, что предлагает рекомендер, уже есть в истории пользователя, что для музыки ожидаемо.

Таблица 3. Базовый рекомендер на 7 593 тестовых пользователях (запуск A).

	Популярное	**GPT, ранжирование**	**GPT, генерация**
NDCG@1	0.0246	0.1655	0.1655
NDCG@5	0.0182	0.1272	0.1329
NDCG@10	0.0186	0.1144	0.1175
Recall@10	0.0114	0.0743	0.0752
MAP@10	0.0079	0.0627	0.0757
HitRate@10	0.1118	0.4465	0.3677
NDCG@10, понравившиеся	0.0146	0.0995	0.1010
Recall@10, пропущенные	0.0108	0.0743	0.0737
Покрытие каталога	0.0002	0.4663	0.5234
Доля уже бывших в истории	0.083	0.421	0.505

2.2 Модель награды

Обе модели награды хорошо предсказывают отложенную обратную связь (Таблица 4) и почти идеально откалиброваны (Рис. 3). Для дальнейшего важны две детали. Во-первых, одно лишь недавнее поведение пользователя — средняя доля по его предыдущим событиям — уже даёт

AUC 0.80, а средняя доля самого трека — только 0.61. Основная часть умения модели — знать, пропускает ли сейчас этот пользователь; знание об отдельных треках — меньшая часть, и именно её оптимизирует RL. Во-вторых, валидационные потери были минимальны после первой эпохи и затем росли, так что используются модели одной эпохи.

Таблица 4. Предсказание доли прослушивания на всех событиях тестового периода, каждое — по событиям до него (запуск A). AUC — для события «прослушано хотя бы наполовину».

Предсказатель доли	**MAE**	**RMSE**	**Кросс-энтропия**	**AUC**
Обучающая модель награды	0.236	0.327	0.433	0.867
Отложенная модель награды	0.243	0.328	0.434	0.865
Среднее по предыдущим событиям пользователя	0.289	0.370	0.510	0.798
Средняя доля трека	0.393	0.429	0.623	0.607
Общее среднее	0.409	0.434	0.635	0.500

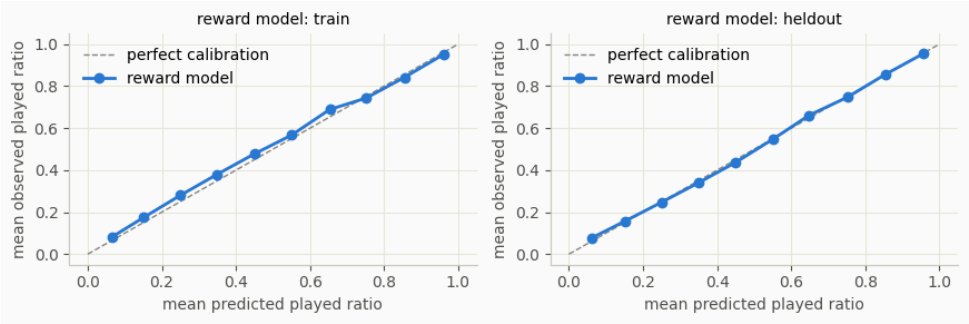


Рис. 3. Калибровка двух моделей награды на событиях тестового периода.

2.3 RL со слабым якорем приводит к коллапсу

В основном запуске вес KL был 0.1. Таблица 5 сравнивает дообученную политику с базовым рекомендером.

Таблица 5. Основной запуск RL в запуске A (вес KL 0.1, 4 000 шагов) против базового рекомендера.

Порождённые списки, тест	**Базовый**	**После RL**	**Изменение**
NDCG@10, все будущие треки	0.1175	0.0125	−89%
NDCG@10, понравившиеся	0.1010	0.0123	−88%
Recall@10, пропущенные	0.0737	0.0059	−92%
Предсказанная доля, обучающая модель награды	0.580	0.811	+40%
Предсказанная доля, отложенная модель награды	0.613	0.713	+16%
Предсказанные секунды, отложенная модель	123.1	93.2	−24%
Покрытие каталога	0.523	0.050	
Доля пользователей, получивших самый частый трек	0.023	0.821	
Ожидаемое пересечение списков двух пользователей	0.001	0.466	
Доля уже бывших в истории	0.505	0.037	
Средняя популярность рекомендованных треков	2 943	757	
Средняя длительность рекомендованных треков (с)	203	130	
Средняя доля рекомендованных треков в логе	0.654	0.906	

Политика действительно максимизировала данную ей награду и перестала рекомендовать. Как это произошло, объясняют четыре наблюдения.

К чему она ушла. У пятнадцати треков, получивших больше всего показов, от 136 до 2 294 прослушиваний в обучающем периоде (у самого популярного трека — 39 487), длительность от 90 до 165 секунд, а их слушатели в среднем дослушивали их на 85–97% против 65% в целом. Самый показываемый из них попал в списки 6 235 из 7 593 пользователей. Это нишевые треки с аудиторией, которая отобрала себя сама: те, кто их нашёл, их любят, а модель награды переносит это на всех. От треков, которые пользователь уже знает, политика отказывается (доля из истории падает с 51% до 4%).

Выигрыш частично получен на ошибках модели. Обучающая модель награды приписывает новым спискам +40%, отложенная — +16%. Отложенная оценка перестаёт расти примерно после 750 шагов, а обучающая продолжает расти до конца (Рис. 5). Всё, что набрано после этой точки, видит только оптимизируемая модель.

Лог выигрыш не подтверждает. Релевантность на понравившихся треках падает так же сильно, как на пропущенных. Политика не научилась избегать того, что пользователи пропускают; она ушла из области, о которой лог что-либо говорит.

Хорошая оценка на отложенных данных модель награды не защитила. Модель точна и откалибрована на событиях, которые породили пользователи. RL спрашивает её о парах «пользователь, трек», которых не было, и там верх берут её представления об объекте в среднем.

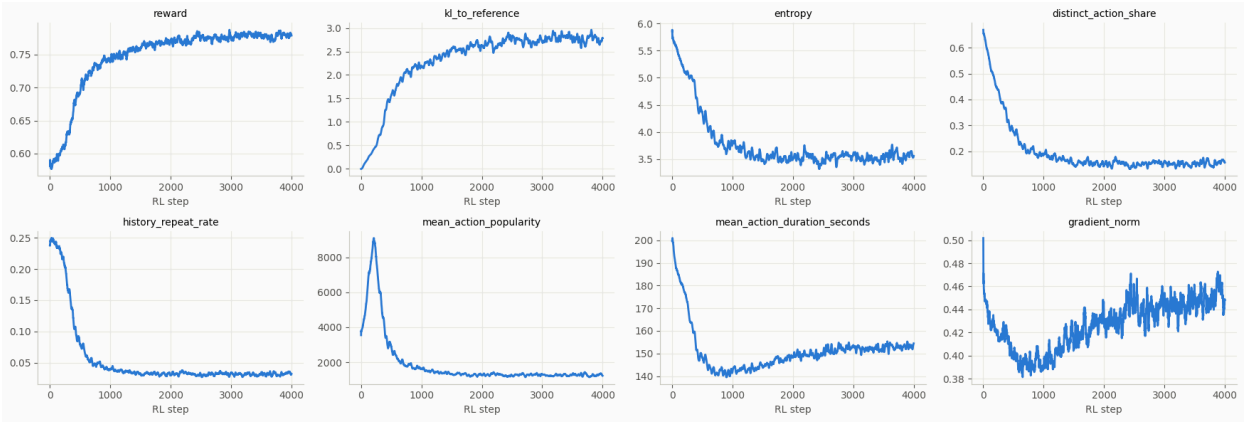


Рис. 4. Что политика порождает во время основного запуска RL в запуске A (скользящее среднее по 20 шагам). KL до базового рекомендера устанавливается около 2.8 ната на позицию; доля различных треков в батче падает с 0.67 до 0.15. Популярность сначала растёт, потом падает намного ниже начального уровня.

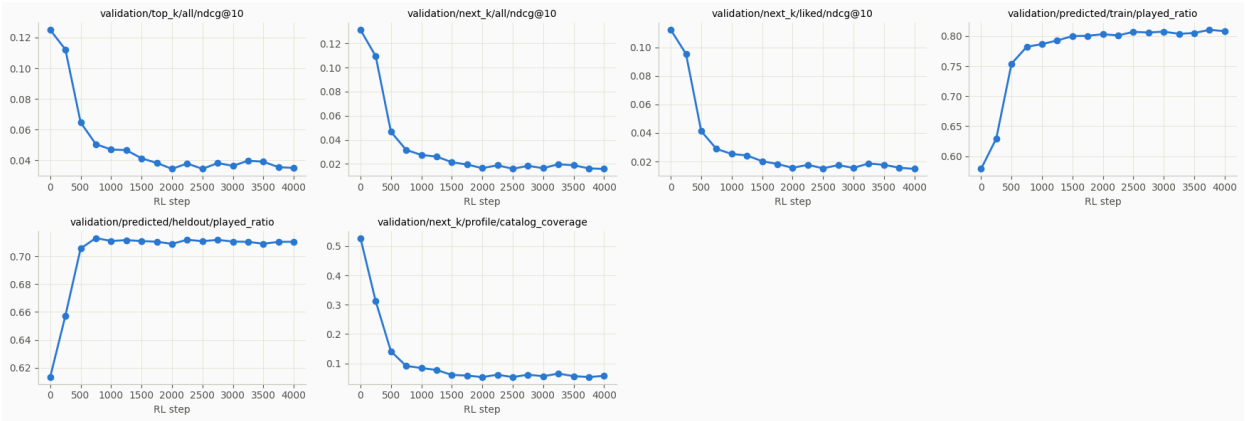


Рис. 5. Жадные списки для валидационных пользователей в том же запуске. Релевантность теряется за первые 500 шагов. Награда по отложенной модели рано выходит на плато; награда по обучающей продолжает расти.

2.4 Якорь решает, и переход резкий

Таблица 6 повторяет дообучение при семи весах KL. Ниже 0.1 политика вырождается полностью: без якоря все пользователи получают одни и те же десять треков (покрытие 0.0002 от 50 000 треков, пересечение 1.0). При 1 и выше она почти не двигается. Интересное поведение лежит между 0.1 и 0.3, и эта сетка его перешагнула.

Таблица 6. Перебор веса KL в запуске A: порождённые списки на тестовых пользователях после 2 000 шагов RL. «Предсказано» — средняя предсказанная доля прослушивания списка.

Вес KL	**Базовый**	**0**	**0.01**	**0.03**	**0.1**	**0.3**	**1**	**3**
NDCG@10, все	0.1175	0.0007	0.0007	0.0006	0.0118	0.1155	0.1183	0.1176
NDCG@10, понравившиеся	0.1010	0.0007	0.0007	0.0005	0.0114	0.0998	0.1021	0.1014
Recall@10, пропущенные	0.0737	0.0002	0.0002	0.0001	0.0059	0.0691	0.0734	0.0734
Предсказано, обучающая	0.580	0.830	0.830	0.836	0.806	0.611	0.591	0.585
Предсказано, отложенная	0.613	0.728	0.728	0.701	0.712	0.633	0.619	0.615
Покрытие каталога	0.523	0.0002	0.0002	0.0004	0.046	0.472	0.514	0.520
Пересечение списков	0.001	1.000	1.000	0.895	0.596	0.005	0.002	0.002
Средняя длительность (с)	203	119	119	123	125	189	199	202
Доля из истории	0.505	0.005	0.005	0.003	0.034	0.479	0.505	0.507

При весе 0.3 получается то, что RL и должен давать. NDCG@10 падает на 1.7%, NDCG на понравившихся — на 1.2%, а Recall на пропущенных — на 6.2%: политика отказывается от пропускаемого больше, чем от слушаемого. Предсказанная доля растёт на 5.3% по обучающей модели награды и на 3.3% по отложенной, то есть большая часть выигрыша подтверждается. Покрытие остаётся 0.47. Рис. 6 показывает тот же перебор как кривую компромисса.

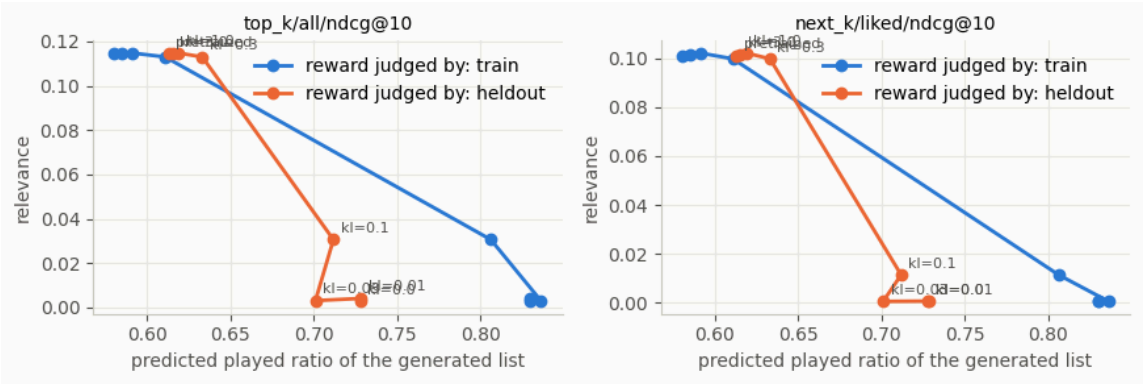


Рис. 6. Релевантность против предсказанной награды в переборе веса KL в запуске A. Расстояние по горизонтали между двумя кривыми — часть награды, о которой сообщает только обучающая модель.

2.5 Каждую награду обходят через свойство, с которым она коррелирует

Были опробованы ещё три определения награды (Таблица 7). Все эти запуски шли с весом KL 0.1, который, как позже показал перебор, лежит на стороне коллапса. Поэтому таблица показывает, в какую сторону обходят каждую награду, а не честное сравнение наград. Его нужно повторить при весе, который держит.

Таблица 7. Варианты награды в запуске A, по 2 000 шагов RL при весе KL 0.1. *Время* умножает долю на относительную длительность трека; *сессия* — предсказанная вероятность того, что пользователь продолжит слушать.

Награда	**Базовый**	**Доля**	**Время**	**Доля + сессия**	**Только сессия**
NDCG@10, все	0.1175	0.0118	0.0003	0.0105	0.0056
Предсказанная доля (отложенная)	0.613	0.712	0.545	0.710	0.634
Предсказанные секунды (отложенная)	123	89	801	89	110
Предсказанное продолжение (отложенная)	0.937	0.941	0.869	0.941	0.941
Покрытие каталога	0.523	0.046	0.0004	0.040	0.025
Пересечение списков	0.001	0.596	0.960	0.555	0.111
Средняя длительность (с)	203	125	1 513	125	180

Награда за долю уводит политику к коротким трекам, которые дослушивают. Награда за время, которая должна была убрать этот стимул, уводит её к 25-минутным трекам и поднимает предсказанные секунды более чем в шесть раз. Награда за продолжение сессии почти не меняет само

предсказанное продолжение (с 0.937 до 0.941): 95% событий и так продолжают сессию, и при промежутке в 30 минут сигналу некуда двигаться. В запуске B, где те же варианты обучались всего 500 меньших шагов, *доля + сессия* и *только сессия* потеряли 17% и 24% NDCG@10, а не почти весь; это говорит о том, что коллапс не успел завершиться, а не о том, что эти награды безопаснее.

2.6 Генерация на несколько шагов вперёд

Рис. 7 прослеживает порождённый список позиция за позицией. У базового рекомендера вероятность того, что порождённый трек есть в реальном продолжении пользователя, падает с 0.165 на первой позиции до 0.06 на десятой, и вместе с ней падает предсказанная моделью награды доля, с 0.67 до 0.55: чем дальше список от реальной истории, тем меньшего ждёт модель награды. У дообученной политики предсказанная доля вдоль списка растёт, с 0.75 до более чем 0.82, а две модели награды с четвёртой позиции расходятся сильнее.

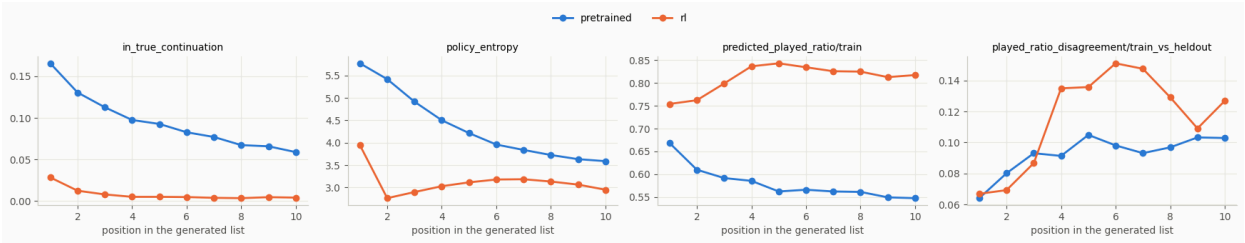


Рис. 7. Порождённые списки по позициям: базовый рекомендер против основного запуска RL в запуске A (тест).

Часть этого роста создаёт сама симуляция. У порождённого трека нет наблюдавшейся обратной связи, но модели награды она нужна, чтобы оценить следующий трек. Рис. 8 оценивает одни и те же списки дообученной политики при трёх допущениях. На десятой позиции предсказанная доля равна 0.82, если модели подаётся её собственное ожидание, 0.91, если случайный исход «пропуск или полное прослушивание», и 0.96, если считать, что каждый порождённый трек прослушан; на первой позиции, которая зависит только от реальной истории, во всех трёх случаях 0.755. Модель награды выучила, что пользователь, только что дослушавший несколько треков, дослушает и следующий, а политика, выбирающая треки с высокой предсказанной долей, сама создаёт себе такой контекст. Поэтому многошаговая предсказанная награда измеряет симуляцию в той же мере, что и рекомендации.

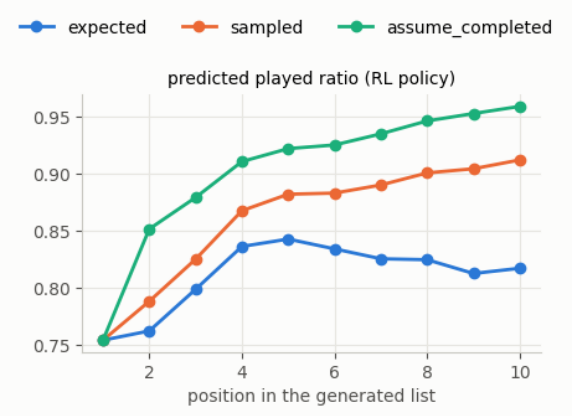


Рис. 8. Предсказанная доля для списков дообученной политики при трёх допущениях об обратной связи на уже порождённые треки (запуск A, обучающая модель награды).

2.7 Запуск B показывает тот же коллапс на более ранней стадии

На первый взгляд запуск B выглядел лучше: его основной запуск RL потерял 68% NDCG@10, а не 89%, и закончился примерно вдвое быстрее на гораздо более слабой видеокарте. Противоречия здесь нет. Он занял меньше времени, потому что сделал гораздо меньше работы: 6 эпох вместо 15 на вдвое меньшем числе обучающих окон с меньшим softmax и 7 запусков перебора по 500 шагов вместо 11 по 2 000. Его политику остановили, пока она ещё разрушалась. Релевантность падает

и на последнем измерении (Рис. 9), а KL до базового рекомендера к концу обучения был 1.7 ната и рос, тогда как в запуске A он вышел на плато 2.8. Его основной запуск увидел примерно в одиннадцать раз меньше порождённых списков.

Перебор запуска B согласуется с запуском A там, где они пересекаются. Уже после 500 шагов запуски с весами KL 0 и 0.03 разрушились (NDCG@10 0.0017 и 0.0043, пересечение списков 1.00 и 0.98), а запуск с 0.3 удержался: NDCG@10 0.1237 против 0.1226 у его базового рекомендера, Recall на пропущенных −2.4%, предсказанная доля +2.7% и +2.4% по двум моделям награды. По абсолютному уровню метрики двух запусков несравнимы: каталоги разные.

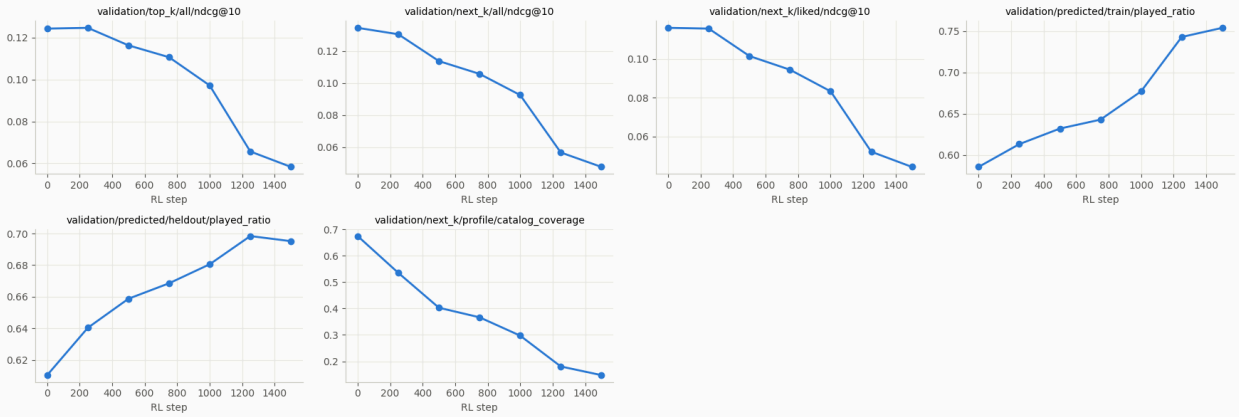


Рис. 9. Запуск B, основной запуск RL: жадные списки для валидационных пользователей. Запуск заканчивается на шаге 1 500, релевантность ещё падает.

3 Yambda-50M, PPO (запуск D)

Запуск D повторяет на Yambda конвейер запуска C: PPO, бейзлайны, альтернативы RL, интервалы. Каталог и данные те же, что в запуске A, так что сравнение с ним отделяет оптимизатор от набора данных.

3.1 Базовый рекомендер среди бейзлайнов

На Yambda GPT-рекомендер — лучший из сравниваемых (Таблица 8): на 4% выше GRU4Rec, на 8% выше BERT4Rec и на 48% выше SASRec. Оговорок три. GRU4Rec и BERT4Rec упёрлись в предел в 10 эпох, когда их валидационные потери ещё снижались, так что они могут быть недообучены. Интервалы между бейзлайнами не считались. Каждая модель обучена один раз. NDCG@10 у SASRec, 0.077, близок к опубликованному для Yambda-50M значению 0.075 [3], хотя протоколы разные.

Таблица 8. Бейзлайны и GPT-рекомендер на 7 593 тестовых пользователях Yambda (запуск D).

	Популярное	**Цепь Маркова**	**GRU4Rec**	**SASRec**	**BERT4Rec**	**GPT, ранж.**	**GPT, генер.**
NDCG@1	0.0246	0.0979	0.1488	0.1030	0.1534	0.1653	0.1653
NDCG@10	0.0186	0.0550	0.1097	0.0767	0.1054	0.1136	0.1165
Recall@10	0.0114	0.0335	0.0718	0.0506	0.0673	0.0737	0.0742
MAP@10	0.0079	0.0272	0.0591	0.0366	0.0585	0.0621	0.0747
HitRate@10	0.1118	0.2612	0.4417	0.3806	0.4163	0.4454	0.3656
NDCG@10, понравившиеся	0.0146	0.0474	0.0976	0.0685	0.0927	0.0989	0.1012
Recall@10, пропущенные	0.0108	0.0371	0.0707	0.0471	0.0678	0.0732	0.0714
Покрытие каталога	0.0002	0.457	0.448	0.397	0.459	0.464	0.519
Доля уже бывших в истории	0.083	0.186	0.426	0.325	0.438	0.423	0.520
Предсказанная доля (отлож.)	0.647	0.608	0.617	0.617	0.613	0.615	0.610

Модели награды совпадают с запуском A (AUC 0.867 и 0.865). Этот запуск впервые показал их качество отдельно по происхождению события: на событиях, поданных рекомендером, модель

точнее (AUC 0.895, MAE 0.198), чем на выбранных пользователем (AUC 0.834, MAE 0.273). RL спрашивает модель именно о рекомендациях, так что это ограничение прошлой версии снято.

3.2 PPO с якорем почти ничего не меняет

При весе KL 0.3 политика на Yambda сдвигается едва заметно (Таблица 9). Все изменения лежат вне своих интервалов, но малы: награда +1.1% по отложенной модели, релевантность +2%.

Таблица 9. Основной запуск RL в запуске D (PPO, вес KL 0.3, 2 000 итераций) против базового рекомендера.

Порождённые списки, тест	**Базовый**	**После RL**	**Изменение**	**95%-й интервал изменения**
NDCG@10, все будущие треки	0.1165	0.1190	+2.2%	от +0.0007 до +0.0043
NDCG@10, понравившиеся	0.1012	0.1032	+2.0%	от +0.0003 до +0.0037
Recall@10, пропущенные	0.0714	0.0745	+4.4%	от +0.0009 до +0.0055
Предсказанная доля, обучающая модель	0.581	0.591	+1.8%	от +0.0097 до +0.0116
Предсказанная доля, отложенная модель	0.610	0.617	+1.1%	от +0.0061 до +0.0079
Предсказанные секунды, отложенная модель	122.3	120.4	−1.6%	
Покрытие каталога	0.519	0.503		
Доля пользователей, получивших самый частый трек	0.026	0.040		
Ожидаемое пересечение списков двух пользователей	0.0014	0.0022		
Доля уже бывших в истории	0.520	0.523		
Средняя популярность рекомендованных треков	3 028	3 382		
Средняя длительность рекомендованных треков (с)	203	197		

Релевантность выросла и на понравившихся, и на пропущенных треках, причём на пропущенных сильнее. Значит, политика не научилась избегать пропусков: она слегка сдвинулась к популярным трекам (популярность +12%), которые чаще попадают в любое продолжение. KL до базового рекомендера за всё обучение дошёл лишь до 0.03 ната на позицию (на KION — 0.10, в запуске A — 2.8). Модель ценности объясняла около 80% дисперсии возвратов против примерно половины на KION: на Yambda награда в основном определяется состоянием пользователя, а не выбранным треком.

3.3 Порог коллапса зависит от оценки преимущества

Таблица 10 повторяет перебор веса KL с PPO. Без якоря политика разрушается так же, как с REINFORCE: все пользователи получают одни и те же примерно 20 треков. Но уже при весе 0.1 она почти не отходит от базовой.

Таблица 10. Перебор веса KL в запуске D: порождённые списки на тестовых пользователях после 1 000 итераций PPO.

Вес KL	**Базовый**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, все	0.1165	0.0011	0.1173	0.1177	0.1175	0.1179	0.1172
NDCG@10, понравившиеся	0.1012	0.0012	0.1020	0.1022	0.1015	0.1020	0.1015
Recall@10, пропущенные	0.0714	0.0002	0.0702	0.0727	0.0725	0.0725	0.0729
Предсказано, обучающая	0.581	0.826	0.598	0.591	0.589	0.588	0.585
Предсказано, отложенная	0.610	0.720	0.625	0.618	0.617	0.614	0.612
Покрытие каталога	0.519	0.0004	0.474	0.498	0.502	0.507	0.512
Пересечение списков	0.0014	1.000	0.0054	0.0026	0.0023	0.0019	0.0017
Средняя длительность (с)	203	122	192	197	198	199	201
Доля из истории	0.520	0.005	0.497	0.517	0.516	0.519	0.520

Таблица 11 ставит два оптимизатора рядом на одних и тех же данных. При весе 0.1 REINFORCE теряет 90% NDCG@10, а PPO не теряет ничего; при 0.3 PPO сдвигает награду втрое слабее.

Резкий переход между 0.1 и 0.3, описанный в разделе 2.4, — свойство той постановки, а не Yambda.

Таблица 11. Изменение относительно базового рекомендера на Yambda: REINFORCE (запуск A, 2 000 шагов) против PPO (запуск D, 1 000 итераций). Награда — предсказанная доля по отложенной модели.

Вес KL	**REINFORCE: NDCG@10**	**REINFORCE: награда**	**PPO: NDCG@10**	**PPO: награда**
0	−99%	+19%	−99%	+18%
0.1	−90%	+16%	+0.7%	+2.5%
0.3	−1.7%	+3.3%	+0.9%	+1.1%
1	+0.7%	+1.0%	+0.6%	+0.4%

Наиболее вероятное объяснение — нормировка преимуществ. В обоих алгоритмах преимущества приводятся к единичной дисперсии, поэтому сила градиента политики фиксирована, а вес KL сравнивается с ней. В REINFORCE базовой линией служил средний возврат других списков для той же истории: вклад пользователя вычитается точно, и нормированное преимущество целиком состоит из различий между действиями. В PPO базовую линию даёт модель ценности, которая объясняет 80% дисперсии; оставшиеся 20% — смесь различий между действиями и ошибки модели ценности. После нормировки полезный сигнал занимает меньшую долю, и тот же вес KL держит политику крепче. Это гипотеза: вместе с оценкой преимущества менялись скорость обучения (втрое ниже) и число списков на шаг. Прямая проверка — PPO с базовой линией из других списков, который в коде предусмотрен.

Практический вывод: вес KL нельзя переносить между постановками, и для PPO на Yambda сетка 0.1–1 лежит целиком на безопасной стороне. Переход находится между 0 и 0.1, и эта сетка его снова не поймала.

3.4 Альтернативы RL

На Yambda все методы в безопасной области дают рост награды на 1–2.5% (Таблица 12, Рис. 10). Дообучение только на понравившемся снова попадает в точку основного запуска RL: NDCG@10 0.118 против 0.119, награда 0.615 против 0.617. Переранжирование покрывает область, до которой RL при этой сетке не дошёл: до +6.5% награды, но ценой релевантности. При равной награде 0.625 порождённые списки RL релевантнее переранжированных (0.117 против примерно 0.113), однако ранжированные списки той же политики дают 0.114. То есть разница — в том, что на Yambda генерация лучше ранжирования, а не в RL.

Таблица 12. RL против дообучения только на понравившемся и переранжирования 100 кандидатов на тесте Yambda (запуск D).

Списки по десять	**NDCG@10, все**	**NDCG@10, понр.**	**Recall@10, проп.**	**Предсказано (отлож.)**	**Покрытие**	**Из истории**
Базовый рекомендер, генерация	0.1165	0.1012	0.0714	0.610	0.519	0.520
Базовый рекомендер, ранжирование	0.1136	0.0989	0.0732	0.615	0.464	0.423
RL, вес KL 1	0.1172	0.1015	0.0729	0.612	0.512	0.520
RL, вес KL 0.5	0.1179	0.1020	0.0725	0.614	0.507	0.519
RL, вес KL 0.3 (основной)	0.1190	0.1032	0.0745	0.617	0.503	0.523
RL, вес KL 0.2	0.1177	0.1022	0.0727	0.618	0.498	0.517
RL, вес KL 0.1	0.1173	0.1020	0.0702	0.625	0.474	0.497
Только на понравившемся	0.1183	0.1029	0.0720	0.615	0.512	0.522
Переранжирование, $w = 1$	0.1141	0.0993	0.0735	0.617	0.461	0.424
Переранжирование, $w = 0.3$	0.1142	0.0998	0.0733	0.621	0.456	0.423
Переранжирование, $w = 0.1$	0.1114	0.0975	0.0713	0.630	0.441	0.408
Переранжирование, $w = 0.03$	0.0941	0.0833	0.0585	0.642	0.400	0.350
Переранжирование, $w = 0$	0.0449	0.0384	0.0281	0.650	0.330	0.226

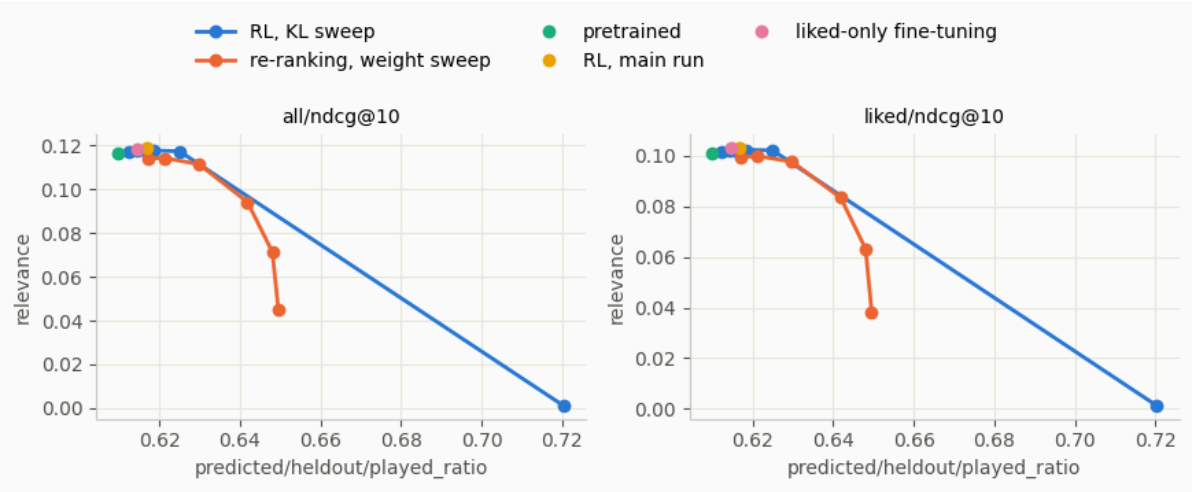


Рис. 10. Релевантность против награды по отложенной модели на Yambda (запуск D). Крайняя правая точка кривой RL — запуск без якоря; между ней и остальными точками сетка весов KL пуста.

3.5 Варианты награды и генерация на несколько шагов

Варианты награды при весе KL 0.3 (Таблица 13) подтверждают то, что запуск A показал при коллапсе. Награда за время уводит к длинным трекам: средняя длительность растёт с 203 до 710 секунд, предсказанное время — втрое, NDCG@10 падает на 22%. Награда за продолжение сессии не сдвигает ничего: предсказанное продолжение 0.9371 против 0.9372. При 95% продолжающихся событий сигналу некуда расти.

Таблица 13. Варианты награды в запуске D, по 1 000 итераций PPO при весе KL 0.3.

Награда	Базовый	Время	Доля + сессия	Только сессия
NDCG@10, все	0.1165	0.0910	0.1173	0.1164
NDCG@10, понравившиеся	0.1012	0.0783	0.1015	0.1006
Предсказанная доля (отлож.)	0.610	0.553	0.614	0.610
Предсказанные секунды (отлож.)	122	364	121	122
Предсказанное продолжение (отлож.)	0.9371	0.9023	0.9372	0.9372
Покрытие каталога	0.519	0.339	0.509	0.513
Пересечение списков	0.0014	0.134	0.0020	0.0016
Средняя длительность (с)	203	710	200	203

Генерация на Yambda по-прежнему лучше ранжирования, и после RL тоже: NDCG@10 0.119 против 0.115. Зависимость предсказанной награды от допущения об обратной связи оказалась ещё сильнее, чем у разрушившейся политики запуска A (Рис. 11): на десятой позиции 0.56, 0.70 или 0.90, разброс 35 пунктов при 0.68 на первой.

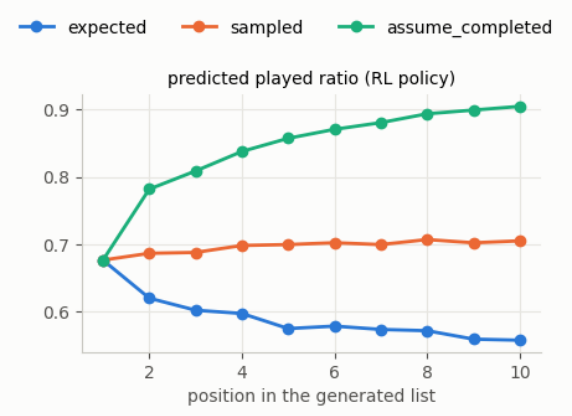


Рис. 11. Предсказанная доля для списков дообученной политики при трёх допущениях об обратной связи на уже порождённые треки (запуск D, обучающая модель награды).

4 KION, PPO (запуск C)

4.1 Базовый рекомендер среди бейзлайнов

Таблица 14 ставит рекомендер рядом с бейзлайнами; все выдают списки из десяти тайтлов, которые пользователь ещё не смотрел. Заметны три вещи.

Популярное на KION — сильная нижняя граница. Десять самых популярных несмотренных тайтлов дают NDCG@10 0.090, и GPT-рекомендер выше на 35%, тогда как на Yambda — в шесть раз. То же видно в соревновании, для которого набор данных был выпущен: пять лучших решений набрали MAP@10 от 0.114 до 0.122 против 0.091 у популярного [4], отношение от 1.25 до 1.34. Здесь отношение по MAP@10 равно 1.37 для GPT-рекомендера и 1.46 для BERT4Rec. Протоколы разные, поэтому сравнимо только отношение, и оно говорит, что рекомендер в ожидаемом диапазоне.

GPT-рекомендер здесь — середина таблицы. Он равен GRU4Rec, на 13% выше SASRec и на 5% ниже BERT4Rec. SASRec отличается от GPT-рекомендера главным образом функцией потерь — один случайный отрицательный пример на позицию вместо полного softmax, — так что разрыв между ними — цена этой функции потерь на этих данных. Разрыв до BERT4Rec, 0.007, в несколько раз больше полуширины бутстрап-интервала, найденной ниже для сравнения с RL (0.001), но между бейзлайнами интервал не считался, и каждая модель обучена один раз.

Списки сосредоточены ещё до RL. Каждая нейросетевая модель рекомендует свой самый частый тайтл примерно половине пользователей, и два пользователя делят от 14% до 21% своих списков. На Yambda те же числа для базового рекомендера — 2% и 0.1%. Поэтому меры коллапса на KION стартуют с высокого уровня.

Генерация списка объект за объектом на KION немного хуже ранжирования (NDCG@10 0.117 против 0.122) и покрывает меньшую часть каталога. Это то, о чём сообщают Петров и Макдональд [1], и противоположно Yambda. Вероятная причина: пользователи Yambda слушают треки повторно, и порождённый трек — правдоподобная часть истории, в которую он попадает, а тайтл на KION смотрят один раз.

Таблица 14. Бейзлайны и GPT-рекомендер на 59 997 тестовых пользователях KION (запуск C). *Досмотренные* — будущие тайтлы, просмотренные хотя бы наполовину; *брошенные* — остальные.

	Популярное	**Цепь Маркова**	**GRU4Rec**	**SASRec**	**BERT4Rec**	**GPT, ранж.**	**GPT, генер.**
NDCG@1	0.0597	0.0669	0.0877	0.0726	0.0945	0.0885	0.0885
NDCG@10	0.0903	0.0959	0.1215	0.1082	0.1290	0.1220	0.1173
Recall@10	0.1303	0.1394	0.1721	0.1580	0.1811	0.1726	0.1620
MAP@10	0.0607	0.0637	0.0824	0.0720	0.0885	0.0831	0.0819
HitRate@10	0.2358	0.2571	0.3154	0.2903	0.3281	0.3144	0.2871
NDCG@10, досмотренные	0.1003	0.1043	0.1294	0.1104	0.1320	0.1293	0.1214
Recall@10, брошенные	0.1078	0.1155	0.1436	0.1346	0.1551	0.1442	0.1382
Покрытие каталога	0.004	0.466	0.544	0.335	0.427	0.574	0.495
Получили самый частый тайтл	0.918	0.530	0.502	0.504	0.496	0.484	0.566
Пересечение списков	0.714	0.220	0.149	0.161	0.161	0.140	0.209
Предсказанная доля (отлож.)	0.473	0.473	0.454	0.452	0.438	0.455	0.444

4.2 Модель награды

Модели награды на KION слабее, чем на Yambda, но заметно лучше простых предсказателей (Таблица 15). Две копии согласованы: корреляция их предсказаний на тестовых событиях 0.94. Калибровка хороша в середине диапазона и сжата по краям: события, предсказанные примерно на 0.06, досматривают до 0.12, а предсказанные примерно на 0.93 — до 0.87.

Для RL важны два свойства. Во-первых, один лишь тайтл предсказывает почти так же хорошо, как недавнее поведение пользователя (AUC 0.66 против 0.67), тогда как на Yambda доминировал пользователь (0.80 против 0.61). На KION награда в большой мере — свойство тайтла, и политика может поднять её выбором тайтлов. Во-вторых, модели завышают долю просмотра для наименее популярной пятой части каталога на 0.13–0.14 против 0.02 для популярных тайтлов. Это та слабость, которой воспользовалась политика на Yambda.

Таблица 15. Предсказание доли просмотра на 184 209 событиях тестового периода KION, каждое — по событиям до него (запуск C). AUC — для события «просмотрено хотя бы наполовину».

Предсказатель доли просмотра	**MAE**	**RMSE**	**Кросс-энтропия**	**AUC**
Обучающая модель награды	0.290	0.347	0.608	0.756
Отложенная модель награды	0.291	0.349	0.610	0.753
Среднее по предыдущим событиям пользователя	0.331	0.384	0.670	0.673
Средняя доля тайтла	0.334	0.379	0.657	0.656
Общее среднее	0.370	0.399	0.689	0.500

4.3 PPO с якорем, который держит

В основном запуске вес KL был 0.3. Таблица 16 сравнивает дообученную политику с базовым рекомендером.

Таблица 16. Основной запуск RL в запуске C (PPO, вес KL 0.3, 2 000 итераций) против базового рекомендера. Интервалы — в абсолютных единицах, парный бутстрап по пользователям.

Порождённые списки, тест	**Базовый**	**После RL**	**Изменение**	**95%-й интервал изменения**
NDCG@10, все будущие тайтлы	0.1173	0.1039	−11.4%	от −0.0144 до −0.0124
NDCG@10, досмотренные	0.1214	0.1228	+1.2%	от 0.0000 до +0.0028
Recall@10, брошенные	0.1382	0.1112	−19.6%	от −0.0289 до −0.0250
Предсказанная доля, обучающая модель	0.433	0.536	+23.9%	от +0.1029 до +0.1041
Предсказанная доля, отложенная модель	0.444	0.545	+22.8%	от +0.1004 до +0.1016
Предсказанные секунды, отложенная модель	6 361	5 632	−11.5%	
Предсказанное продолжение, отложенная модель	0.804	0.806	+0.3%	
Покрытие каталога	0.495	0.434		
Доля пользователей, получивших самый частый тайтл	0.566	0.554		
Ожидаемое пересечение списков двух пользователей	0.209	0.275		
Средняя популярность тайтлов (зрителей)	37 483	40 467		
Средняя длительность тайтлов (часов)	5.7	3.2		
Средняя доля рекомендованных тайтлов в логе	0.457	0.560		

Это тот результат, который метод и должен давать. Пять наблюдений его поддерживают или уточняют.

Выигрыш подтверждён. Отложенная модель награды сообщает +22.8% там, где обучающая — +23.9%, и обе оценки растут вместе и выходят на плато примерно после 500 итераций (Рис. 12). Запуск перебора с тем же весом, остановленный на 1 000 итераций, заканчивается в той же точке. Здесь нет ничего, что видела бы только оптимизируемая модель.

Лог согласен. Релевантность на тайтлах, которые пользователь затем посмотрел хотя бы наполовину, не изменилась, и её интервал начинается в нуле. Релевантность на брошенных тайтлах падает на пятую часть. Политика отказывается от того, что бросают, и сохраняет то, что досматривают.

К чему она ушла. Больше всего показов получили фильмы длиной от 85 до 130 минут со средней долей просмотра от 0.54 до 0.76; лидер роста перешёл из списков 8 388 пользователей в списки 26 169 из 59 997. Больше всего потеряли сериалы: один, с 17 часами содержимого и средней долей 0.08, упал с 33 948 списков до 8 045, другой — с 16 278 до

558. Средняя длительность рекомендованного тайтла упала с 5.7 часа до 3.2. Выросли и два нишевых фильма с 953 и 175

зрителями в обучающем периоде, до 20 403 и 15 901 списка; это та же картина, что на Yambda, в малой дозе.

Награда говорит меньше, чем кажется. Доля просмотра сериала считается от всего сериала, поэтому сериал редко оказывается «досмотренным» по правилу половины и получает низкую оценку у всех. И рост награды, и расхождение между досмотренными и брошенными тайтлами во многом пересказывают один факт: политика теперь предпочитает фильмы. Предсказанное время просмотра падает на 11.5%. Ценнее ли досмотренный фильм, чем начатый сериал, — продуктовый вопрос, на который доля просмотра отвечает по построению.

Как вёл себя PPO. Политика осталась близко к базовому рекомендеру: KL установился около 0.10 ната на позицию против 2.8 в запуске A. Доля действий за пределами порога обрезки всё время была нулевой, то есть обрезка ни разу не сработала; каждое обновление удерживали

внутри порога малая скорость обучения и якорь. Модель ценности объясняла около половины дисперсии возвратов. Фактически это метод «актёр-критик» с обобщённой оценкой преимущества и четырьмя проходами по каждому батчу.

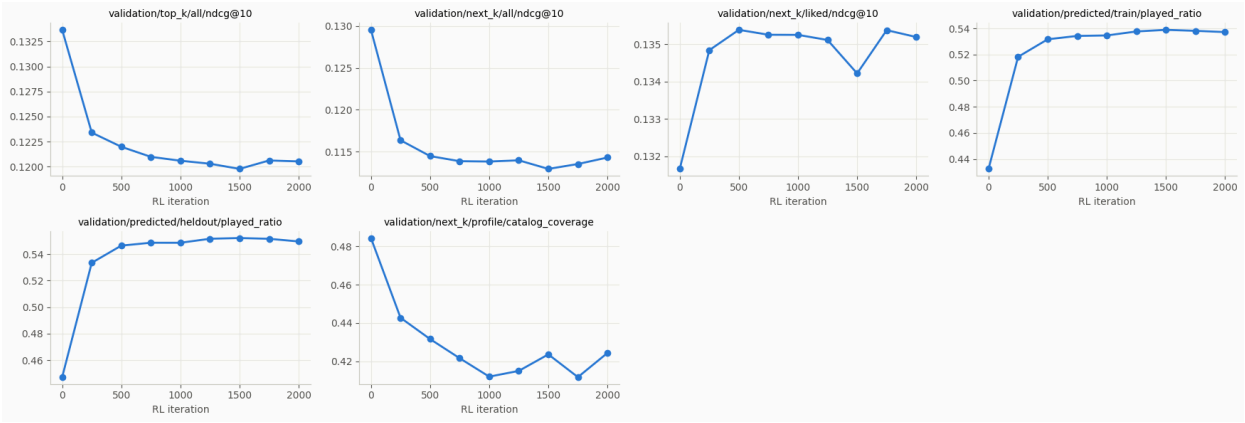


Рис. 12. Запуск C, основной запуск RL: жадные списки для валидационных пользователей. За первые 500 итераций релевантность на всех тайтлах падает, а на досмотренных растёт; тогда же выходит на плато награда по обеим моделям.

4.4 Компромисс плавный

Таблица 17 повторяет дообучение при шести весах KL, Рис. 13 рисует его кривой.

Таблица 17. Перебор веса KL в запуске C: порождённые списки на тестовых пользователях после 1 000 итераций PPO. «Предсказано» — средняя предсказанная доля просмотра списка.

Вес KL	**Базовый**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, все	0.1173	0.0007	0.0736	0.0987	0.1042	0.1114	0.1153
NDCG@10, досмотренные	0.1214	0.0012	0.0944	0.1206	0.1240	0.1261	0.1250
Recall@10, брошенные	0.1382	0.0004	0.0654	0.1006	0.1099	0.1280	0.1350
Предсказано, обучающая	0.433	0.871	0.653	0.559	0.534	0.499	0.467
Предсказано, отложенная	0.444	0.748	0.652	0.570	0.544	0.509	0.477
Предсказанные секунды (отлож.)	6 361	4 511	4 101	5 675	5 775	6 006	6 098
Покрытие каталога	0.495	0.006	0.248	0.386	0.423	0.451	0.474
Пересечение списков	0.209	0.991	0.377	0.317	0.280	0.250	0.226
Средняя длительность (часов)	5.7	1.7	1.9	3.0	3.3	4.1	4.8

Каждое ослабление якоря покупает предсказанную награду ценой релевантности, и ни один шаг не является обрывом. От веса 1 до 0.2 выигрыш в награде растёт с 8% до 28%, а релевантность на досмотренных тайтлах остаётся в пределах 4% от базовой, чаще выше; теряется релевантность на брошенных, от 2% до 27%. Ниже 0.2 уходят и досмотренные: при 0.1 награда выше на 47%, NDCG@10 на досмотренных ниже на 22%, общий NDCG@10 — на 37%. Без якоря политика разрушается, как на Yambda: рекомендуется всего 46 тайтлов, и два пользователя делят 99% своих списков.

Две модели награды согласны при всех весах, кроме нуля, где обучающая сообщает 0.87, а отложенная — 0.75. Использование ошибок модели награды появляется только на краю.

При весе KL 0.1 политика на Yambda потеряла 89% NDCG@10, а на KION — 37%. Заслугой PPO это считать нельзя: вместе с ним изменились набор данных, скорость обучения и число обновлений, а PPO без якоря разрушается точно так же.

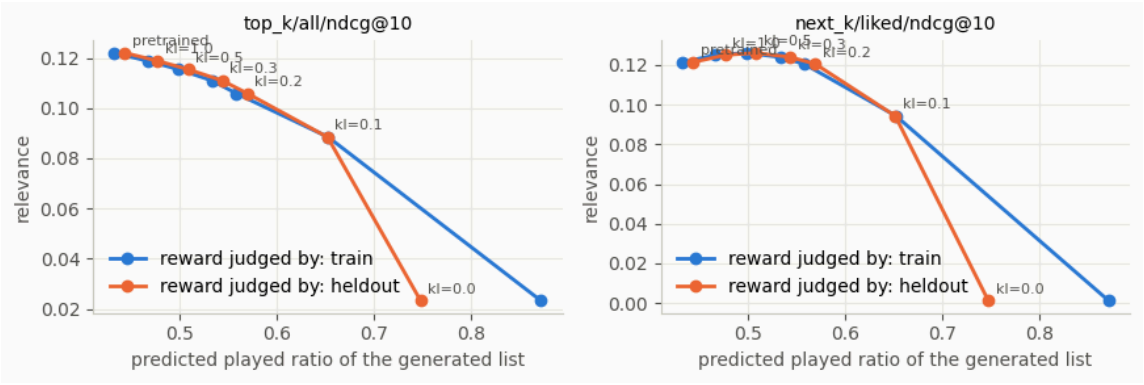


Рис. 13. Релевантность против предсказанной награды в переборе веса KL в запуске C. Две кривые совпадают везде, кроме нулевого веса.

4.5 Нужен ли RL

Таблица 18 ставит два более дешёвых метода рядом с запусками RL, а Рис. 14 рисует всё на плоскости «релевантность — награда по отложенной модели».

Таблица 18. RL против дообучения только на понравившемся и переранжирования 100 кандидатов на тесте KION (запуск C). Списки RL и дообучения порождаются; переранжирование стартует с ранжирования базового рекомендера.

Списки по десять	**NDCG@10, все**	**NDCG@10, досм.**	**Recall@10, брош.**	**Предсказано (отлож.)**	**Покрытие**	**Длит. (часов)**
Базовый рекомендер, генерация	0.1173	0.1214	0.1382	0.444	0.495	5.7
Базовый рекомендер, ранжирование	0.1220	0.1293	0.1442	0.455	0.574	6.2
RL, вес KL 1	0.1153	0.1250	0.1350	0.477	0.474	4.8
RL, вес KL 0.5	0.1114	0.1261	0.1280	0.509	0.451	4.1
RL, вес KL 0.3 (основной)	0.1039	0.1228	0.1112	0.545	0.434	3.2
RL, вес KL 0.2	0.0987	0.1206	0.1006	0.570	0.386	3.0
RL, вес KL 0.1	0.0736	0.0944	0.0654	0.652	0.248	1.9
Только на понравившемся	0.1036	0.1258	0.1017	0.528	0.473	3.0
Переранжирование, $w = 1$	0.1198	0.1335	0.1377	0.485	0.567	4.6
Переранжирование, $w = 0.3$	0.1110	0.1335	0.1194	0.534	0.530	3.2
Переранжирование, $w = 0.1$	0.0927	0.1211	0.0878	0.592	0.435	2.4
Переранжирование, $w = 0.03$	0.0722	0.0977	0.0667	0.620	0.346	2.1
Переранжирование, $w = 0$	0.0545	0.0745	0.0528	0.626	0.329	2.0

Переранжирование не уступает RL до награды примерно 0.59. При $w = 0.3$ оно даёт предсказанную долю 0.534 при NDCG@10 0.111 в целом и 0.134 на досмотренных. У ближайшего по награде запуска RL (основной, 0.545) — 0.104 и 0.123; у ближайшего по релевантности (вес 0.5, NDCG@10 0.111) награда 0.509. При $w = 1$ переранжирование улучшает порождённые списки базового рекомендера сразу по всем осям. Переранжированные списки покрывают большую часть каталога, чем списки RL при той же награде, но дают самый частый тайтл большему числу пользователей (0.58 при $w = 0.3$ и 0.67 при $w = 0.1$ против 0.55 у основного запуска RL). Часть преимущества — стартовая точка: переранжирование начинает с ранжированного списка, который на KION на 0.005 NDCG@10 лучше порождённого. Ранжированные списки самой

политики RL дают 0.110 в целом и 0.130 на досмотренных, наравне с переранжированием при $w = 0.3$, но их предсказанная награда не измерялась.

Дообучение только на понравившемся попадает в точку основного запуска RL. Одна эпоха обычного обучения на объектах, просмотренных хотя бы наполовину, даёт тот же общий NDCG@10 (0.104), чуть более высокую релевантность на досмотренных, более низкий Recall на брошенных и предсказанную награду 0.528 против 0.545. Модель награды для этого не нужна. Эта стадия вместе со всем переранжированием заняла 3.4 минуты; основной запуск RL — 14.3.

RL достаёт дальше. Переранжирование 100 кандидатов не может поднять предсказанную долю выше 0.626, и возле этого предела его релевантность резко падает. RL при весе KL 0.1 достигает 0.652 при NDCG@10 0.074, что выше, чем 0.055–0.072 у переранжирования возле предела. Это единственная область, где RL — лучший инструмент, и это область, где релевантность на досмотренных уже ниже на 22%.

Вероятная причина — награда. Предсказанная доля просмотра — сумма слагаемых по отдельным объектам, и на KION она в основном свойство тайтла. Тогда выбирать объекты с высокой предсказанной наградой по одному почти оптимально, а это и делает переранжирование. RL должен выигрывать, когда награда зависит от списка в целом, а у этой награды такая зависимость есть только через симулированную обратную связь на предыдущие объекты.

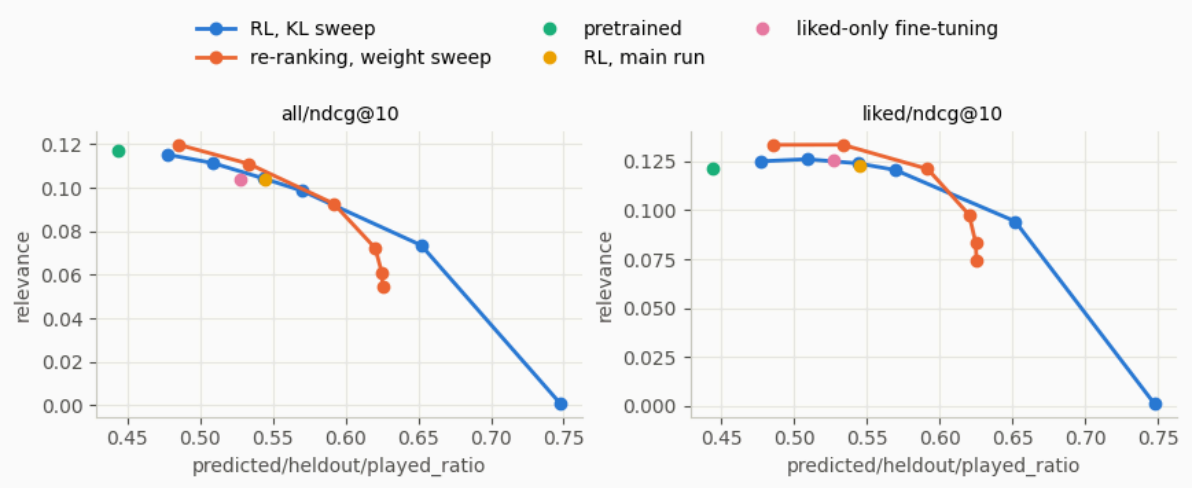


Рис. 14. Релевантность против награды по отложенной модели на KION (запуск C). Крайняя правая точка кривой RL — запуск без якоря.

4.6 Варианты награды при якоре, который держит

На Yambda варианты награды сравнивались при весе, ведущем к коллапсу. Здесь все они шли при 0.3, и ни один не разрушился (Таблица 19).

Таблица 19. Варианты награды в запуске C, по 1 000 итераций PPO при весе KL 0.3. *Время* умножает долю на относительную длительность тайтла; *сессия* — предсказанная вероятность того, что пользователь посмотрит ещё один тайтл в течение 7 дней.

Награда	Базовый	Доля	Время	Доля + сессия	Только сессия
NDCG@10, все	0.1173	0.1042	0.0885	0.1098	0.1093
NDCG@10, досмотренные	0.1214	0.1240	0.0901	0.1217	0.1131
Recall@10, брошенные	0.1382	0.1099	0.1042	0.1281	0.1324
Предсказанная доля (отлож.)	0.444	0.544	0.307	0.487	0.440
Предсказанные часы (отлож.)	1.77	1.60	10.51	1.62	1.61
Предсказанное продолжение (отлож.)	0.804	0.805	0.763	0.812	0.821
Покрытие каталога	0.495	0.423	0.274	0.487	0.505
Пересечение списков	0.209	0.280	0.304	0.189	0.144
Средняя длительность (часов)	5.7	3.3	63.6	4.3	5.1

Награда за время просмотра, которая должна была убрать предпочтение коротких тайтлов, промахивается в другую сторону, как и на Yambda. Политика уходит к длинным сериалам, средняя длительность рекомендованного тайтла растёт с 5.7 часа до 64, предсказанное время просмотра — в шесть раз, а предсказанная доля, продолжение и четверть релевантности теряются. Добавление сессии к доле вдвое уменьшает сдвиг: доля растёт на 10% вместо 23%, релевантность на досмотренных не меняется, общий NDCG@10 падает на 6%. Награда только за сессию поднимает предсказанную вероятность ещё одного просмотра в течение недели с 0.804 до 0.821 и оставляет долю на месте. Она стоит 7% NDCG@10, на досмотренных столько же, сколько на всех, и это единственный вариант, при котором списки становятся менее похожими: пересечение двух пользователей падает с 0.21 до 0.14, покрытие растёт. У недельного сигнала возвращения больше запаса, чем у 30-минутного на Yambda, но 1.7 пункта за 7% релевантности — слабый рычаг.

4.7 Генерация на несколько шагов вперёд

Рис. 15 прослеживает порождённый список по позициям. Вероятность того, что порождённый тайтл есть в реальном продолжении пользователя, падает с 0.088 на первой позиции до 0.016 на десятой у базового рекомендера и с 0.081 до 0.018 после RL. Генерация отстаёт от ранжирования по мере роста списка, и после RL сильнее (Таблица 20).

Таблица 20. Ранжирование против генерации объект за объектом на тесте KION (запуск C). Первый объект совпадает по построению.

NDCG на	Базовый, ранж.	Базовый, генер.	RL, ранж.	RL, генер.
1	0.0885	0.0885	0.0812	0.0812
5	0.1055	0.1044	0.0943	0.0911
10	0.1220	0.1173	0.1103	0.1039

Предсказанная доля просмотра падает вдоль списка у обеих политик: с 0.53 до 0.38 у базового рекомендера и с 0.62 до 0.43 после RL. Преимущество дообученной политики сжимается с десяти пунктов на первой позиции до пяти на десятой. Две модели награды расходятся на 0.052–0.057 на каждой позиции у обеих политик; RL не уводит список в область, где они расстаются.

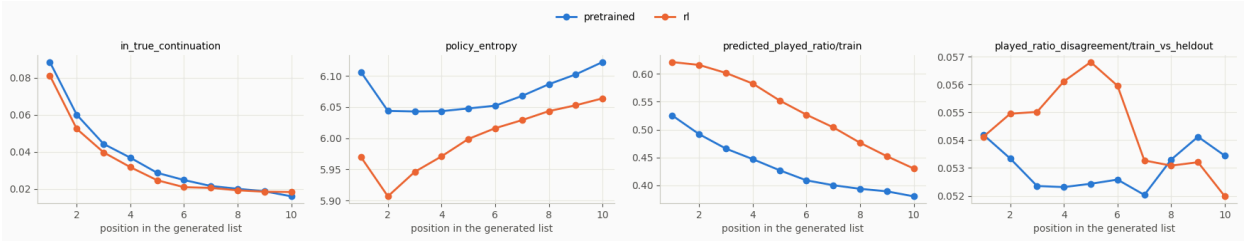


Рис. 15. Порождённые списки по позициям: базовый рекомендер против основного запуска RL в запуске C (тест).

Как и на Yambda, значительную часть профиля вдоль списка создаёт симуляция (Рис. 16). На первой позиции предсказанная доля дообученной политики равна 0.62 при любом допущении. На десятой она равна 0.43, если модели награды подаётся её собственное ожидание для ранее порождённых тайтлов, 0.53 — если случайный исход, и 0.68 — если считать каждый порождённый тайтл досмотренным: разброс в 25 пунктов. Ожидание здесь — самое пессимистичное допущение, и причина, вероятно, во входе, который оно создаёт. Реальные доли просмотра близки к 0 или к 1; историю из тайтлов, каждый из которых просмотрен примерно наполовину, модель награды почти не видела. RL обучался с этим допущением, поэтому уровень его многошаговой награды и спад вдоль списка (Рис. 15) — отчасти его артефакты.

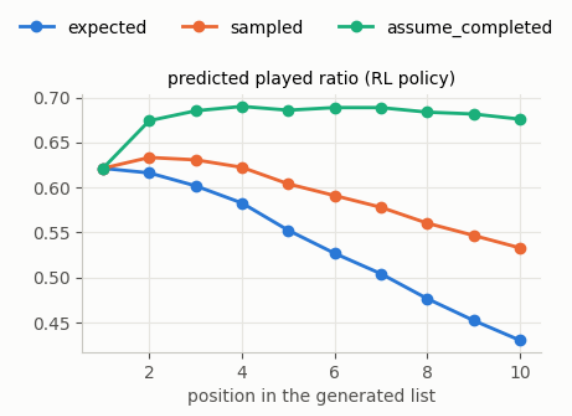


Рис. 16. Предсказанная доля просмотра для списков дообученной политики при трёх допущениях об обратной связи на уже порождённые тайтлы (запуск C, обучающая модель награды).

5 KION, вторая итерация (запуск E)

5.1 Что изменилось и воспроизводится ли запуск C

В данных вместо одного сигнала «сессия продолжается» теперь два: *продолжение сессии* (следующее событие не позже, чем через промежуток g) и *возвращение* (новая сессия начинается в пределах окна w); формулы — в «Теоретическом минимуме», раздел 4.4. На KION сессия — один календарный день, возвращение — просмотр в другой день в течение недели. Сессию продолжают 42% событий, возвращение следует за 71%. Исход, который лог не успел показать, теперь считается неизвестным и не участвует ни в обучении, ни в оценке; исход возвращения известен для 89% событий. Порядок тайтлов внутри дня на KION — порядок строк в файле, поэтому «продолжение сессии» говорит скорее о том, сколько тайтлов было за день, чем о том, какой из них удержал зрителя.

Модель награды получила два новых выхода: возвращение $\hat{u}$ и вероятность выбора $\hat{q}$. Предсказание доли от этого не пострадало: AUC 0.753 и 0.755 против 0.756 и 0.753 в запуске C.

Базовый рекомендер и бейзлайны совпали с запуском C до последнего знака. Модели награды обучены заново, так что повтор основного запуска RL — проверка на втором экземпляре модели награды (Таблица 21). Результат воспроизвёлся; интервал для NDCG@10 на досмотренных теперь не содержит нуля (от +0.0003 до +0.0030).

Таблица 21. Основной запуск RL и два запуска перебора в запусках C и E. Рекомендер один и тот же, модели награды обучены заново.

Основной запуск RL, вес KL 0.3	**Запуск C**	**Запуск E**
NDCG@10, все	0.1039 (−11.4%)	0.1039 (−11.4%)
NDCG@10, досмотренные	0.1228 (+1.2%)	0.1230 (+1.3%)
Recall@10, брошенные	0.1112 (−19.6%)	0.1103 (−20.2%)
Предсказанная доля, обучающая модель	+23.9%	+24.0%
Предсказанная доля, отложенная модель	+22.8%	+23.1%
Покрытие каталога	0.434	0.438
Пересечение списков	0.275	0.260
Средняя длительность тайтлов (часов)	3.2	3.3
Вес KL 0.1: NDCG@10, все	0.0736	0.0767
Вес KL 0.1: предсказанная доля, отложенная	+46.9%	+40.6%
Без якоря: тайтлов во всех списках	46	81

5.2 Удержание предсказуемо, но выбором объекта почти не управляется

Оба сигнала предсказываются заметно лучше случайного (Таблица 22). Для возвращения в тестовом периоде известны исходы только 26 тыс. событий из 184 тыс., потому что окно в неделю равно длине периода; основное свидетельство — валидационный период.

Таблица 22. Качество предсказания сигналов удержания обучающей моделью награды (запуск E). У отложенной модели числа отличаются не больше чем на 0.004.

Сигнал	Период	AUC	Частота	Событий с известным исходом
Продолжение сессии	валидация	0.660	0.427	187 398
Продолжение сессии	тест	0.661	0.421	184 209
Возвращение	валидация	0.696	0.720	187 398
Возвращение	тест	0.719	0.702	26 004

Предсказуемость ещё не значит, что сигнал годится как награда. Таблица 23 показывает для 100 лучших кандидатов каждого тестового пользователя, насколько каждый сигнал различается между кандидатами одного пользователя — это всё, что может использовать политика, — и между пользователями. Доля просмотра различается между кандидатами так же сильно, как между пользователями. Возвращение — вчетверо слабее: 0.03 против 0.13. Вернётся ли пользователь, зависит в основном от пользователя, а не от того, что ему показать.

Таблица 23. Сигналы обучающей модели награды на 100 кандидатах каждого тестового пользователя (запуск E). Разброс — стандартное отклонение. *AUC релевантности* — вероятность того, что у кандидата, который пользователь реально потребил следующим, значение выше, чем у остальных.

Сигнал	Разброс между кандидатами	Разброс между пользователями	AUC релевантности
Доля просмотра $\hat{\rho}$	0.155	0.157	0.535
Продолжение сессии $\hat{c}$	0.051	0.101	0.505
Возвращение $\hat{u}$	0.031	0.126	0.546
Выбор $\hat{q}$	0.141	0.116	0.560
$\hat{q} \cdot \hat{\rho}$	0.128	0.145	0.541
Логарифм вероятности по рекомендеру	0.700	0.154	0.701

5.3 Голова выбора отличает подмену, но о релевантности не знает

Голова выбора выполняет задачу, на которой обучалась: AUC для реального объекта против подменённого 0.70 у обучающей модели и 0.68 у отложенной; средняя оценка реальных объектов 0.70, подменённых 0.53. Но как сигнал релевантности она не работает, и это видно четырьмя способами.

- **Среди кандидатов рекомендера.** AUC релевантности у $\hat{q}$ — 0.56, у логарифма вероятности по самому рекомендеру — 0.70 (Таблица 23). О том, что пользователю подходит, рекомендер знает гораздо больше.
- **На разрушившихся политиках.** Политика без якоря с NDCG@10 0.001 получает по отложенной модели $\hat{q} \cdot \hat{\rho} = 0.53$ против 0.34 у базового рекомендера; сама $\hat{q}$ падает лишь с 0.74 до 0.70. Величина, которая должна была падать при потере релевантности, растёт.
- **На самой релевантной политике.** Политика с наградой из лога (следующий раздел) сохраняет NDCG@10 и получает самую низкую $\hat{q}$: от 0.60 до 0.71.
- **Вдоль списка.** $\hat{q}$ растёт с 0.73 на первой позиции до 0.80 на десятой, тогда как вероятность попасть в реальное продолжение падает с 0.088 до 0.016 (Рис. 17).

Причина, скорее всего, в устройстве отрицательных примеров. Подмены берутся с частотой популярности, поэтому $\hat{q}$ измеряет, насколько объект вероятнее для этого пользователя, чем в среднем по всем. Популярный тайтл получает около 0.5, кто бы его ни смотрел. А на KION релевантность в значительной мере и есть популярность: популярное даёт 0.090 из 0.122. Голова выбора штрафует как раз то, что чаще всего оказывается релевантным. К тому же её обучение

не сошлось: AUC на валидации рос все восемь эпох (с 0.63 до 0.72), а остановка произошла по общим потерям.

Дообучение по награде $\hat{q} \cdot \hat{\rho}$ это подтверждает (Таблица 24): при каждом весе якоря релевантность ниже, чем у простой доли, а при весе 0.1 политика теряет 70% NDCG@10 против 35%.

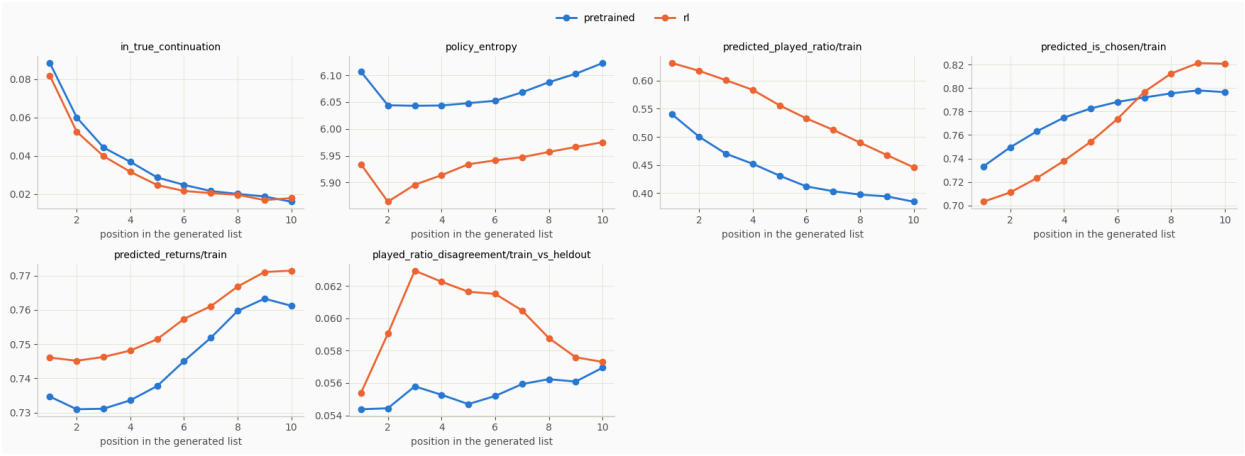


Рис. 17. Порождённые списки по позициям в запуске E: базовый рекомендер против основного запуска RL. Вероятность выбора и возвращения по модели награды растут вдоль списка, а попадание в реальное продолжение падает.

5.4 Награда из лога удерживает релевантность без якоря, но не разнообразие

Награда статьи [1] — попадание порождённого объекта в реальное продолжение пользователя с весом $1/\log_2(k+1)$ — проверялась без якоря: одна и в сумме с предсказанной долей просмотра (Таблица 24, Рис. 18).

Таблица 24. Награды с собственным слагаемым релевантности в запуске E, по 1 000 итераций PPO. Строки с долей $\hat{\rho}$ — перебор веса KL из того же запуска.

Награда	**Вес KL**	**NDCG@10, все**	**NDCG@10, досм.**	**Recall@10, брош.**	**Доля (отлож.)**	**Покрытие**	**Пересечение**
Базовый рекомендер		0.1173	0.1214	0.1382	0.459	0.495	0.209
Доля $\hat{\rho}$	0.3	0.1048	0.1239	0.1123	0.562	0.434	0.260
	0.1	0.0767	0.0980	0.0686	0.645	0.274	0.284
	0	0.0010	0.0015	0.0005	0.743	0.010	0.987
Доля, взвешенная выбором, $\hat{q} \cdot \hat{\rho}$	0.3	0.0994	0.1141	0.1105	0.539	0.447	0.156
	0.1	0.0352	0.0465	0.0244	0.672	0.238	0.477
	0	0.0004	0.0006	0.0002	0.687	0.004	0.999
Только релевантность по логу	0	0.1180	0.1279	0.1417	0.474	0.097	0.519
Доля + 10 × релевантность по логу	0	0.0971	0.1229	0.0843	0.640	0.103	0.577
Доля + 3 × релевантность по логу	0	0.0749	0.0970	0.0622	0.698	0.088	0.498
Доля + 1 × релевантность по логу	0	0.0088	0.0137	0.0035	0.718	0.013	0.964
Доля + 0.3 × релевантность по логу	0	0.0042	0.0066	0.0020	0.714	0.012	0.984

Одна релевантность по логу держит релевантность без всякого якоря. NDCG@10 +0.6%, на досмотренных +5.4%. Заметного выигрыша в NDCG нет, как и в статье [1], где такое дообучение дало тот же NDCG, что обычное обучение. Но списки стянулись к популярному: покрытие каталога упало с 0.50 до 0.10, пересечение списков двух пользователей выросло с 0.21 до 0.52, а на 1% самых популярных тайтлов теперь приходится 91% рекомендаций вместо 57%. Метрики релевантности этого не видят.

В сумме с долей просмотра она удерживает релевантность лучше якоря. При весе 10 предсказанная доля растёт на 39%, NDCG@10 на досмотренных остаётся прежним (+1%), общий падает на 17%. Якорь при той же награде (вес KL 0.1, +41%) теряет 19% на досмотренных и 35% в целом. При весе 3 награда растёт на 52% при потере 36% NDCG@10; якорь в эту область не попадает вовсе. При весе 1 и ниже слагаемое релевантности перестаёт удерживать, и политика разрушается.

Разнообразия она не удерживает. Во всех запусках с релевантностью по логу покрытие около 0.10, тогда как у якоря при той же награде — 0.27. Якорь сохраняет всё распределение базового рекомендера, включая его разнообразие; награда из лога сохраняет только попадания, а попасть проще всего популярным. Два механизма дополняют друг друга, и их естественно сочетать.

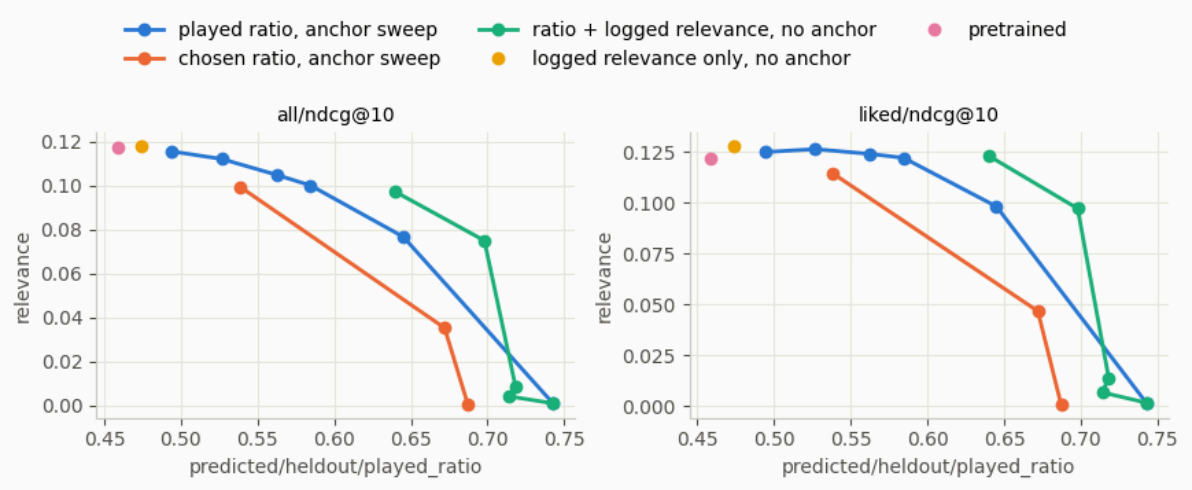


Рис. 18. Релевантность против предсказанной доли по отложенной модели в запуске E. Доля с релевантностью по логу без якоря лежит выше перебора якоря; доля, взвешенная выбором, — ниже.

5.5 Награды за удержание

Четыре награды с сигналами удержания шли при трёх весах якоря; Таблица 25 приводит два из них.

Таблица 25. Награды за удержание в запуске E, по 1 000 итераций PPO. Доля, продолжение и возвращение — предсказания отложенной модели награды для порождённых списков.

Награда	**NDCG@10, все**	**NDCG@10, досм.**	**Доля $\hat{\rho}$**	**Продолжение $\hat{c}$**	**Возвращение $\hat{u}$**	**Покрытие**	**Пересечение**
Базовый рекомендер	0.1173	0.1214	0.459	0.348	0.713	0.495	0.209
Вес KL 0.3							
Доля	0.1048	0.1239	0.562	0.343	0.728	0.434	0.260
Сессия	0.0999	0.1017	0.439	0.397	0.739	0.558	0.107
Возвращение	0.1088	0.1130	0.462	0.368	0.732	0.517	0.147
Доля + возвращение	0.1093	0.1211	0.510	0.356	0.729	0.488	0.188
Доля + сессия + возвращение	0.1072	0.1160	0.494	0.367	0.733	0.519	0.156
Вес KL 0.1							
Доля	0.0767	0.0980	0.645	0.382	0.745	0.274	0.284
Сессия	0.0245	0.0287	0.161	0.609	0.769	0.385	0.334
Возвращение	0.0903	0.0945	0.466	0.393	0.760	0.551	0.091
Доля + возвращение	0.0952	0.1139	0.559	0.365	0.749	0.467	0.186
Доля + сессия + возвращение	0.0874	0.1020	0.534	0.395	0.764	0.536	0.113

Награда за возвращение — слабый рычаг, как и предсказывал разброс сигнала. Она поднимает предсказанную вероятность возвращения на 1.9 пункта (с 0.713 до 0.732) за 7% NDCG@10 при весе 0.3 и на 4.7 пункта за 23% при весе 0.1. При этом обычная награда за долю просмотра сама поднимает её на 1.5 пункта при весе 0.3: по модели награды, чаще возвращаются те, кто досмотрел. Отдельная награда за возвращение добавляет к этому меньше половины пункта.

Награда за продолжение сессии вредна. При весе 0.1 предсказанное продолжение растёт с 0.35 до 0.61, а предсказанная доля просмотра падает с 0.46 до 0.16 и предсказанное время просмотра — почти в шесть раз. Политика рекомендует тайтлы, которые бросают: после брошенного тайтла человек в тот же день включает другой, и сессия «продолжается». На KION этот сигнал поощряет неудовлетворённость.

Возвращение в награде делает списки разнообразнее. Пересечение списков двух пользователей падает с 0.21 до 0.15 при весе 0.3 и до 0.09 при 0.1, покрытие растёт. В сумме с долей возвращение работает как тормоз: при весе 0.1 «доля + возвращение» теряет 19% NDCG@10 вместо 35% у простой доли, но и доля растёт на 22% вместо 41%.

Все значения возвращения здесь — предсказания модели, обученной на наблюдениях: она знает, что после таких объектов люди возвращались чаще, но не знает, что причина в объекте (Рис. 19).

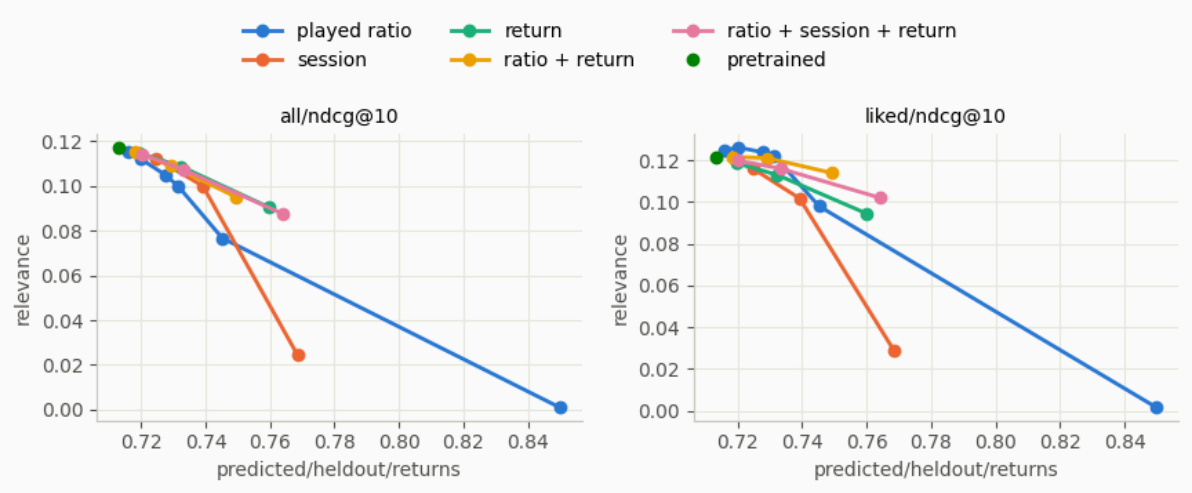


Рис. 19. Релевантность против предсказанной вероятности возвращения в запуске E; каждая линия — одна награда при трёх весах якоря. Крайняя правая точка — простая доля без якоря: разрушившаяся политика получает самую высокую предсказанную вероятность возвращения.

5.6 Методы статьи [2] и остальные альтернативы

Таблица 26 и Рис. 20 ставят рядом с RL четыре способа поднять награду без порождения списков в цикле обучения.

Таблица 26. RL и альтернативы на тесте KION (запуск E). κ — порог весов значимости.

Списки по десять	**NDCG@10, все**	**NDCG@10, досм.**	**Recall@10, брош.**	**Доля (отлож.)**	**Покрытие**	**Пересечение**
Базовый рекомендер, генерация	0.1173	0.1214	0.1382	0.459	0.495	0.209
RL, вес KL 0.3 (основной)	0.1039	0.1230	0.1103	0.565	0.438	0.260
Только на понравившемся	0.1036	0.1258	0.1017	0.552	0.473	0.242
REINFORCE вне политики, $\kappa = 0.5$	0.1032	0.1237	0.1039	0.538	0.504	0.205
REINFORCE вне политики, $\kappa = 2$	0.0901	0.1136	0.0854	0.564	0.424	0.197
REINFORCE вне политики, $\kappa = 10$	0.0575	0.0737	0.0529	0.576	0.320	0.153
Переранжирование, $w = 1$	0.1199	0.1336	0.1375	0.503	0.564	0.147
Переранжирование, $w = 0.3$	0.1119	0.1343	0.1200	0.554	0.534	0.168
Переранжирование, $w = 0.1$	0.0944	0.1235	0.0883	0.610	0.449	0.197
Переранжирование, $w = 0$	0.0568	0.0782	0.0544	0.639	0.373	0.170
Лучший из 2 списков	0.1079	0.1125	0.1357	0.479	0.506	0.195
Лучший из 8 списков	0.1053	0.1133	0.1296	0.507	0.496	0.197
Лучший из 32 списков	0.1021	0.1110	0.1259	0.526	0.498	0.199

REINFORCE вне политики при пороге 0.5 попадает в точку основного запуска RL и не теряет разнообразия. NDCG@10 тот же (0.103), на досмотренных +1.9%, награда +17% против +23% у RL. При этом покрытие и пересечение списков остались как у базового рекомендера (0.504 и 0.205), тогда как RL сужает списки (0.438 и 0.260). Модель награды этому методу не нужна. Подъём порога дорог: при 10 теряется 51% NDCG@10 ради +26% награды. Направление то же, что в [2], где подъём порога с 0.5 до 10 стоил 9% Recall, но эффект здесь намного сильнее. До нескольких объектов политика не сходится ни при каком пороге: при 10 покрытие 0.32.

Отбор лучшего из нескольких списков слабее всех. Случайный выбор из четырёх самых вероятных объектов уже стоит релевантности (0.108 при двух списках против 0.117 у жадного списка), а награда растёт медленно: +4% при двух списках, +15% при тридцати двух. Переранжирование и RL при той же награде релевантнее.

Переранжирование по-прежнему не уступает RL до награды около 0.62, то есть до роста примерно на треть; дальше достаёт только RL.

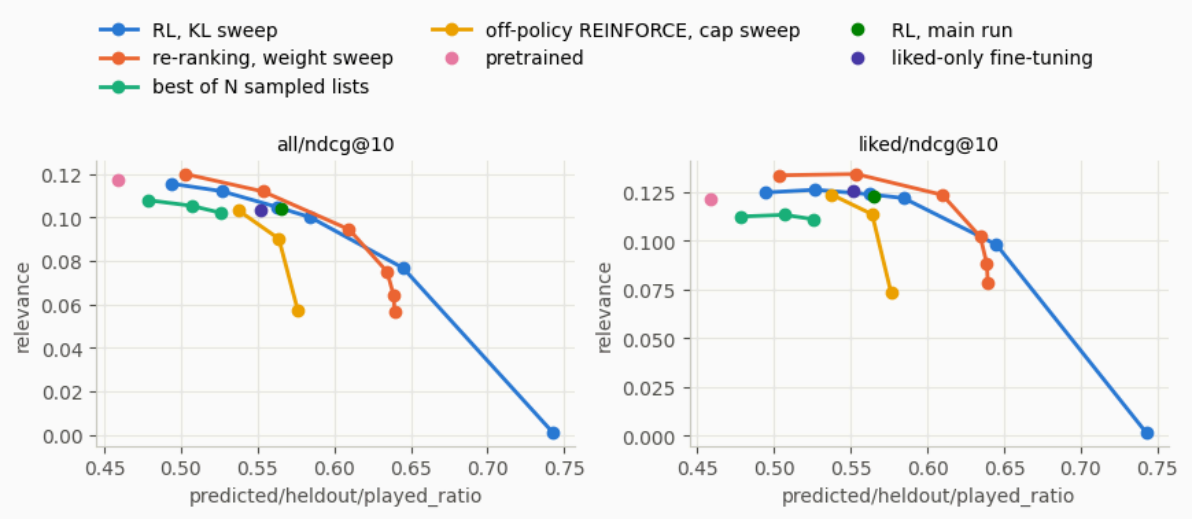


Рис. 20. Релевантность против награды по отложенной модели на KION (запуск E): RL, переранжирование, REINFORCE вне политики и отбор лучшего из нескольких списков.

6 Сравнение с двумя статьями

Статьи сообщают об RL, который сохраняет релевантность. Здесь первый основной запуск её не сохранил, запуск на KION сохранил ценой 11% общего NDCG@10, а запуск PPO на Yambda почти ничего не изменил. Отличие от статей не в оптимизаторе, а в том, откуда берётся награда и на каких данных обновляется политика (Таблица 27).

Таблица 27. Чем различаются три постановки. Числа для [1] — NDCG@10 на MovieLens-1M и Steam-2M по их настройкам; числа для [2] — их таблицы 1 и 3. Числа этой работы — по отложенной модели награды.

	Петров и Макдональд [1]	**Матвеев и соавторы [2]**	**Эта работа**
Награда	NDCG по реальному следующему объекту плюс разнообразие или штраф за популярность	Залогированные лайк и доля прослушивания залогированных рекомендаций	Доля, предсказанная обученной моделью
На чём обновляется политика	На собственных порождённых списках	На залогированных сессиях, веса значимости ограничены 0.5	На собственных порождённых списках
Что удерживает релевантность	Слагаемое NDCG внутри награды	Политика может только перевзвешивать залогированные действия	KL-штраф к базовому рекомендеру
Роль обученной модели награды	Нет	Только оценка	Обучающий сигнал
Релевантность после RL	NDCG@10 от +3% до −38% в зависимости от веса второго слагаемого	Recall@100 −2%; −6% на действиях с наименьшим возвратом	NDCG@10 при весе KL 0.3: на Yambda −2% с REINFORCE и +2% с PPO, на KION −11%. При 0.1: −89%, +1% и −35%
Награда после RL	Целевая метрика улучшается	Возврат по модели +2% … +4%	Предсказанная доля при 0.3: на Yambda +3% с REINFORCE и +1% с PPO, на KION +23%

Отсюда три вывода. Во-первых, в режиме, похожем на их, — с сильным якорем — результат на Yambda совпадает с Матвеевым и соавторами почти число в число: потеря релевантности около 2%, потеря примерно втрое больше на взаимодействиях, которых пользователи не хотели, и

выигрыш в награде в несколько процентов. KION показывает ту же форму в большем масштабе: −11% в целом, −20% на брошенных тайтлах, без потери на досмотренных, +23% награды. Во-вторых, обе статьи тоже теряют релевантность, когда ограничение ослаблено. Петров и Макдональд теряют 15% NDCG@10 на MovieLens-1M и 38% на Steam-2M при самом сильном штрафе за популярность; Матвеев и соавторы теряют 9% Recall, когда порог весов значимости поднят с 0.5 до 10. Чего у них не может случиться — это коллапса к десяти объектам: у одних релевантность по реальным данным стоит в награде, у других политика не покидает залогированных действий. Постановка этой работы, где обученная модель — единственный источник награды, снимает обе защиты. Матвеев и соавторы делают ту же оговорку о собственных числах по модели: результаты, отобранные и оценённые одной и той же моделью обратной связи, не являются независимым свидетельством. В-третьих, оптимизатор Петрова и Макдональда от коллапса не защищает: PPO без якоря разрушился так же, как REINFORCE, а с якорем его обрезка ни разу не сработала. Изменился только вес якоря, при котором начинается коллапс (раздел 3.3).

Вторая итерация проверила защиты обеих статей напрямую, на KION (раздел 5). Награда из [1] действительно удерживает релевантность без якоря и, как у авторов, не даёт заметного прироста NDCG; в сумме с предсказанной долей она удерживает релевантность лучше якоря. Но в их статье не видно того, что видно здесь: без ограничения на разнообразие политика стягивается к популярному. Метод из [2] тоже ведёт себя как описано: при пороге 0.5 сдвиг мягкий и разнообразие сохраняется, с ростом порога релевантность теряется, а до нескольких объектов политика не сходится никогда. Обе защиты работают, и каждая защищает своё: награда из лога — попадания, обучение на логе — распределение.

7 Пять вопросов: что известно сейчас

1. **Меняется ли поведение и максимизируется ли награда?** Да, и тем сильнее, чем слабее ограничение. Политика следует за свойством объектов, с которым награда коррелирует: на Yambda — к коротким малоизвестным трекам, на KION — от сериалов к фильмам, при награде за время — к самым длинным объектам, при награде за продолжение сессии — к тайтлам, которые бросают. На KION отложенная модель подтверждает выигрыш при любом ненулевом весе якоря; на Yambda с REINFORCE — меньше половины. Сколько политика сдвинется, зависит и от того, насколько награда определяется объектом, а не пользователем: модель ценности, которая очередного объекта не видит, объясняет половину разброса возвратов на KION и 80% на Yambda, и там PPO при том же якоре почти не двигается.
2. **Сохраняется ли релевантность?** Тремя способами, и каждый сохраняет своё. Якорь сохраняет распределение базового рекомендера: на KION при весе 0.3 общий NDCG@10 падает на 11% целиком за счёт брошенных тайтлов, разнообразие сужается умеренно. Награда из лога сохраняет попадания даже без якоря, но не разнообразие: покрытие падает впятеро. Обучение на залогированных действиях сохраняет и то, и другое, но даёт меньший выигрыш. Без всякого ограничения все пользователи получают одно и то же. Попытка научить саму модель награды релевантности — голова выбора — не удалась.
3. **Долгосрочные цели.** Возвращение в течение недели на KION предсказуемо (AUC 0.70–0.72), но зависит в основном от пользователя: между кандидатами одного пользователя сигнал различается вчетверо слабее, чем между пользователями. Награда за него даёт 1.9 пункта предсказанной вероятности за 7% релевантности, и почти столько же даёт обычная награда за долю просмотра. Продолжение сессии как награда вредно на KION и неподвижно на Yambda. Офлайн можно проверить предсказуемость сигнала, наличие у него рычага и действие на списки; вернутся ли пользователи на самом деле, покажет только онлайн-тест.
4. **Несколько шагов вперёд.** Списки с каждой позицией отходят от реального продолжения; на KION генерация хуже ранжирования, на Yambda лучше. Предсказанная награда поздних позиций зависит от допущения об обратной связи: до 35 пунктов на Yambda и около 26 на

KION. Сигналы модели награды вдоль списка растут, пока реальные попадания падают. Многошаговую оценку награды из этого симулятора нельзя приводить без этой оговорки.

5. **Компромисс.** Он есть везде, но его форма зависит от того, чем удерживается релевантность. С якорем на KION он плавный. С наградой из лога кривая лежит выше: больше награды при той же релевантности, меньше разнообразия. Переранжирование той же моделью награды даёт ту же или лучшую кривую до роста награды примерно на треть, так что в этой области компромисс принадлежит модели награды и рекомендеру, а не RL. Вес якоря между постановками не переносится: одна и та же величина означает коллапс с REINFORCE и почти полную неподвижность с PPO.

8 Ограничения

- Одно начальное значение на конфигурацию. Запуск E повторил основной результат на KION с другими моделями награды, но с тем же рекомендером. Интервалы покрывают выборку пользователей, а не разброс между запусками обучения. У бейзлайнов интервалов нет.
- Объяснение разницы между REINFORCE и PPO на Yambda — гипотеза: вместе с оценкой преимущества менялись скорость обучения и число списков на шаг.
- Исследование наград проведено только на KION. Запуск D шёл на предыдущей версии кода: без сигналов удержания, без наград из лога, без REINFORCE вне политики и без бейзлайна «повтор истории».
- На Yambda с PPO сетка весов KL не захватила переход между неподвижностью и коллапсом; он лежит между 0 и 0.1.
- GRU4Rec и BERT4Rec на Yambda остановлены пределом эпох, а не валидационными потерями. Бейзлайны не настраивались.
- Голова выбора обучена на подменах с частотой популярности и остановлена до сходимости; её неудача говорит об этой конструкции, а не обо всех способах научить модель награды релевантности.
- Исход возвращения в тестовом периоде известен только для 14% событий; качество этого сигнала оценено в основном на валидационном периоде, по которому выбиралась и эпоха остановки.
- Обрезка PPO почти не срабатывала (на KION ни разу, на Yambda для 0.01% действий), так что ни один запуск не проверяет ту часть PPO, которая его отличает.
- Переранжирование стартует с ранжированных списков, RL оценивается по порождённым; на KION это в пользу переранжирования, на Yambda — в пользу RL.
- На KION доля просмотра привязана к длине тайтла, а вместе с ней — награда и деление на досмотренные и брошенные.
- Преимущества нормировались, чего нет ни в одной из двух статей; веса KL несравнимы ни с их шкалой, ни между нашими постановками.
- Всё офлайн. Ни одно число здесь не говорит, как реальные пользователи отреагировали бы на дообученную политику.

9 Что дальше

- **Награда из лога вместе со слабым якорем.** Первая удерживает попадания, второй — разнообразие; по отдельности каждая теряет своё. Это главный кандидат на метод, который лучше и якоря, и переранжирования.
- **Проверить гипотезу о нормировке:** PPO с базовой линией из других списков той же истории при тех же весах KL на Yambda. Заодно пройти веса между 0 и 0.1.
- **Исследование наград на Yambda.** Там награда сильнее определяется пользователем, чем объектом, и вывод о слабом рычаге может оказаться ещё жёстче.
- **Другая голова выбора или отказ от неё:** отрицательные примеры из кандидатов самого рекомендера, а не по популярности, и остановка по её собственному качеству.

- **Несколько начальных значений** для рекомендера, бейзлайнов и основного запуска RL.
- **Награда без привязки к длине:** доля просмотра относительно обычной для тайтла или его типа.

Литература

1. A. V. Petrov, C. Macdonald. Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning. arXiv:2403.04875, 2024.
2. A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, K. Khrylchenko. Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction. arXiv:2607.02818, 2026.
3. A. Ploshkin et al. Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval. arXiv:2505.22238, 2025.
4. A. Petrov, I. Safilo, D. Tikhonovich, D. Ignatov. MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation. arXiv:2209.00325, 2022. Результаты соревнования: README репозитория `github.com/irsafilo/KION_DATASET`.

Приложение: на что ушло время

Таблица 28. Время работы по стадиям. Итерация PPO из 512 списков занимает 1.1 секунды на Yambda (каталог 50 000, окна по 128) и 0.43 секунды на KION (каталог 7 992, окна по 48).

Стадия, минут	**A**	**B**	**C**	**D**	**E**
Базовый рекомендер	99	109	4	66	4
Модели награды	6	22	1	6	4
Бейзлайны			5	81	6
Основной запуск RL	74	56	14	38	14
Перебор веса KL	249	61	33	109	33
Альтернативы RL			3	6	16
Перебор наград	142	46	16	54	65
Релевантность внутри награды					43
Всё остальное	1	6	3	1	3
Всего	570	300	79	361	188